

Higher Private School of Engineering and Technology



Academic Project Report
PIDEV — 2025 / 2026



Where Skills Meet Projects
Smart Freelance & Project Matching Platform

Realized by :

Full Name	Role
Nour Jbeli	Full-Stack Developer
Moatez Mathlouthi	Full-Stack Developer
Safa Hajji	Full-Stack Developer
Louay Krouna	Full-Stack Developer
Oumaima Ouerfelli	Full-Stack Developer

Information	Details
Supervisor	Mrs. Nadine MAAZOUNE , Mrs. Leila BEN DHIEF and Mrs. Hela Nefla
Field	4th Year Software Engineering — SAE
Academic Year	2025 – 2026
Project Type	Integrated Development Project (PIDEV)
Technologies	Spring Boot · Angular · Docker · Jenkins · Kubernetes · SonarQube

Table of Contents

Table of Contents.....	3
List of Figures.....	5
List of Tables.....	7
Acknowledgements	8
General Introduction.....	9
Chapter 1 : Project Context	11
Introduction	11
1. Project Presentation	11
1.1 General Presentation.....	11
1.2 Workify Services	11
2. Study of Existing Solutions	12
2.1 Comparative Study	12
International Platforms	12
Local Platforms (Tunisia).....	13
2.2 Critical Analysis of Existing Solutions	13
2.3 Problem Statement.....	13
3. Proposed Solution.....	14
4. Analysis and Choice of the Working Method	14
4.1 Definition of the Scrum Method.....	14
4.2 Choice of Methodology	15
Conclusion.....	16
Chapter 2 : Requirements Analysis and Specification	17
Introduction	17
1. Identification of Actors.....	17
1.1 Administrator.....	17
1.2 Freelancer	17
1.3 Client (Project Owner).....	18
1.4 Partner.....	18
1.5 Service Desk	18
2. Functional Requirements.....	18
3. Non-Functional Requirements.....	19
4. Product Backlog	20
5. Sprint Planning	22
6. Use Case Identification.....	22
6.1 Global Use Case Diagram	22

6.2 Class Diagram.....	23
Conclusion.....	24
Chapter 3 : Conceptual Study and Implementation.....	25
Introduction	25
1. Sprint 0 : Analysis and Specification	25
1.1 Sprint Backlog	25
1.2 Global Use Case Diagram	Erreur ! Signet non défini.
1.3 Class Diagram.....	Erreur ! Signet non défini.
1.4 Global Architecture	26
2. Sprint 1 : User Management, Project & Offer, AI Matching	26
2.1 Sprint Backlog	26
Implementation — User Management	28
Implementation — Project & Offer Management.....	29
AI Matching — Criteria and Weights	31
3. Sprint 2 : Communication, Feedback, Events, Opportunities, Tickets & Tasks	32
3.1 Sprint Backlog	32
Implementation — Message & Conversation	33
Implementation — Feedback & Response	35
3.6 Implementation — Event & Sponsor	37
3.7 Implementation — Opportunity & Request	37
3.8 Implementation — Ticket & Support.....	Erreur ! Signet non défini.
3.9 Implementation — Task & Planning.....	39
4. Sprint 3 : Training, Certification, Subscriptions & DevOps	43
4.1 Sprint Backlog	43
Implementation — Certificate & Training	45
Implementation — Pack & Subscription.....	52
4.6 DevOps Pipeline.....	59
5. Versioning with GitHub	61
5.1 Branching Strategy	61
6. Work Environment	63
6.1 Hardware Environment.....	63
6.2 Software Environment.....	63
Conclusion.....	65
General Conclusion	66
Bibliography	68

List of Figures

Figure	Title	Page
Figure 1	Agile Scrum methodology	
Figure 2	Global use case diagram	
Figure 3	Global class diagram	
Figure 4	User registration and login interface	
Figure 5	Admin dashboard — User management interface	
Figure 6	AI Avatar generation interface	
Figure 7	Project list and creation interface	
Figure 8	Offer submission and comparison interface	
Figure 9	AI Matching results — Recommended freelancers interface	
Figure 10	Real-time messaging interface	
Figure 11	Conversation priority and auto-archive system	
Figure 12	Feedback submission and list interface	
Figure 13	Event creation and management interface	
Figure 14	Internship opportunity list and application interface	
Figure 15	Support ticket creation and service desk dashboard	
Figure 16	Task management and calendar view interface	
Figure 17	Training list and details interface	

Figure	Title	Page
Figure 18	Generated PDF certificate with QR code	
Figure 19	Certificate verification page (public QR code link)	
Figure 20	Use case diagram — Pack and Subscription	
Figure 21	Pack selection and subscription interface	
Figure 22	Online payment and PDF invoice interface	
Figure 23	Jenkins pipeline — All 8 stages passed	
Figure 24	SonarQube dashboard — Quality Gate Passed	
Figure 25	Kubernetes dashboard — Pods running in production	
Figure 26	GitHub repository — Branches and commit history	
Figure 27	GitHub Pull Requests — Review and merge workflow	
Figure 28	GitHub Actions / Jenkins CI triggered on push	

List of Tables

Table	Title	Page
Table 1	Project general information	
Table 2	Workify functional modules	
Table 3	Comparative study — International platforms	
Table 4	Comparative study — Local platforms (Tunisia)	
Table 5	Product Backlog	
Table 6	Sprint planning	
Table 7	Sprint 0 backlog	
Table 8	Technology stack	
Table 9	Sprint 1 backlog	
Table 10	AI Matching criteria and weights	
Table 11	Sprint 2 backlog	
Table 12	Sprint 3 backlog	
Table 13	Jenkins CI/CD pipeline stages	
Table 14	Git branching strategy	
Table 15	Commit convention	
Table 16	Hardware environment	
Table 17	Software environment	

Acknowledgements

We would like to express our sincere gratitude to Mrs. Nadine MAAZOUNE, our supervisor, for her regular follow-up, valuable advice and availability throughout the realization of this project. Her guidance helped us better orient our technical choices and maintain a serious and structured work pace.

We would also like to thank Mrs. Leila BEN DHIEF and Mrs. Hela Nefla for their constructive feedback and valuable directions during the evaluation sessions, which helped us improve the quality of our work at every stage.

We would like to express our appreciation to all the teachers of the Software Engineering department at ESPRIT for the quality of the education they have provided us throughout this academic year. The knowledge and skills acquired in the various subjects — web development, databases, software architecture and DevOps — formed the essential foundation for the completion of this project.

Finally, we would like to thank our families and loved ones for their continuous support and encouragement throughout this academic journey.

General Introduction

The rapid growth of the digital economy and the widespread adoption of remote work have led individuals and organizations to increasingly rely on online platforms to manage their professional collaborations. In this context, the freelancing sector is no exception. The creation of platforms dedicated to connecting freelancers with clients has become essential to optimize talent sourcing, project management and secure payment processing within a transparent and efficient ecosystem.

This report presents our academic project carried out within the PIDEV module at the Higher Private School of Engineering and Technology (ESPRIT), where we contributed to the design, development and implementation of a web platform dedicated to the intelligent matching of freelancers and projects. Throughout this period, we had the opportunity to work as a team of five members to address the specific needs of both freelancers and clients through an intelligent and integrated solution. This report is structured into several chapters.

The first chapter, entitled "Project Context", describes the general framework of our project. It includes a presentation of the project, a critical analysis of existing solutions, a comparative study and a problem statement. We then propose the solution that will be developed, specifying the design and development methodologies adopted.

The second chapter, entitled "Requirements Analysis and Specification", presents the functional and non-functional requirements of the project. This chapter also addresses the analysis of these requirements based on the UML modeling language. The last section of this chapter is dedicated to the definition of the product backlog and the project management using the Scrum methodology, including the identification of the global use case diagram.

The third chapter, entitled "Conceptual Study and Implementation", is dedicated to the presentation of the project sprints. This chapter covers the detailed design, development and testing of each sprint, as well as the versioning strategy adopted using GitHub. The last part of this chapter is devoted to the presentation of the hardware and software environment used for the implementation of our platform.

We close our report with a general conclusion that summarizes the different stages that led to the successful completion of this work.

Chapter 1 : Project Context

Introduction

In this first chapter, we place our project in its general context. We begin with a presentation of the academic framework in which this project was carried out, as well as the objectives targeted and the different services offered by our platform.

In the second section of this chapter, we analyze the current situation and the problems surrounding the freelancing domain. Subsequently, we propose our solution and present the objectives we aim to achieve, as well as the development methodology adopted.

1. Project Presentation

1.1 General Presentation

This section aims to present the Workify project carried out as part of the PIDEV module at ESPRIT.

Information	Details
Project Name	Workify — Where Skills Meet Projects
Type	Integrated Development Project (PIDEV)
Institution	ESPRIT — Higher Private School of Engineering and Technology
Academic Year	2025 – 2026
Supervisor	Mrs. Nadine MAAZOUNE
Team	5 fourth-year Software Engineering students

Table 1 : Project general information

Workify is an intelligent and modular web platform dedicated to freelance management. It connects freelancers, clients, partners and administrators within a unified digital ecosystem, covering the entire lifecycle of a freelance collaboration : from project publication to skills certification.

1.2 Workify Services

Workify provides all platform actors with an integrated and complementary set of services across ten functional modules :

Module	Description
User & Access Management	Registration, JWT authentication, RBAC, AI avatar generation
Project & Offer Management	Project publication, offer submission, AI matching, lifecycle tracking
Task & Planning	Task CRUD, calendar view (day/week/month), automated reminders
Message & Conversation	Real-time messaging, auto-archive, AI inappropriate content detection
Feedback & Response	Bidirectional evaluations, fraud detection, AI sentiment analysis
Event & Sponsor	Event creation, sponsor management, Google Calendar, Twilio notifications
Opportunity & Request	Internship publication, application management, status tracking
Ticket & Support	Support tickets, auto-assignment, prioritization, resolution tracking
Certificate & Training	Online training, automatic PDF certificate generation, QR code verification
Pack & Subscription	Subscription management, online payment, PDF invoicing, AI recommendation

Table 2 : Workify functional modules

2. Study of Existing Solutions

In order to introduce the proposed solution, we present a comparative study of existing freelancing platforms, both at the international level and at the local Tunisian level.

2.1 Comparative Study

International Platforms

Platform	Advantages	Limitations
Upwork	Large audience, secure payment system, detailed profiles	High fees, keyword-based matching only, no integrated training
Freelancer.com	Bidding system, contest organization	Complex interface, lack of trust, no structured communication
Fiverr	Clear service catalog, easy to use	Not suited for complex projects, no artificial intelligence
Malt	Verified profiles, structured European market	Geographically limited, no integrated training or certification

Table 3 : Comparative study — International platforms

Local Platforms (Tunisia)

Platform	Advantages	Limitations
Freelances.tn	Project publication, freelancer profiles	No intelligent matching, very limited communication
Keejob	Structured recruitment-oriented	Not suited for project-based freelancing, no contract management
JobTalent	Structured profiles	Does not support freelance workflows, no integrated evaluation

Table 4 : Comparative study — Local platforms (Tunisia)

2.2 Critical Analysis of Existing Solutions

Before proposing our solution, we draw up a critical analysis of existing platforms in order to clearly identify the problems faced by users :

- Insufficient matching algorithms : existing platforms use keyword-based searches without semantic analysis or artificial intelligence, generating poorly relevant recommendations and forcing clients to manually browse through numerous profiles.
- Fragmented communication : most platforms do not have a structured internal messaging system. Users are forced to rely on external tools such as email or WhatsApp, which undermines the traceability of exchanges.
- Unreliable evaluation systems : existing evaluation mechanisms are vulnerable to abuse and fraud. There is no automatic detection system for suspicious feedback nor sentiment analysis of reviews.
- Absence of training and certification : none of the analyzed platforms integrates a professional training program with generation of verifiable certificates, thus limiting the skills development of freelancers.
- No event management : existing platforms do not offer a dedicated module for organizing professional events or managing sponsors.
- Rigid monetization : existing business models are inflexible and do not adapt to the specific needs of each type of user.
- Unstructured customer support : there is no integrated ticketing system allowing users to report problems and track their resolution in an organized manner.

2.3 Problem Statement

The findings established through the study of existing solutions reveal that current freelancing platforms suffer from significant fragmentation of their functionalities. The various actors face recurring unresolved challenges :

- How to improve the relevance of matching between freelancers and projects by integrating artificial intelligence and semantic analysis of skills ?
- How to ensure secure and structured internal communication between the different actors of the platform ?
- How to guarantee the reliability of evaluations and automatically detect fraudulent behavior ?
- How to integrate a certifying training program directly within the platform to promote professional development ?
- How to offer a flexible and intelligent monetization model that adapts to the needs of each user ?
- How to centralize all these services within a single, modular and scalable solution ?

3. Proposed Solution

To address all the identified challenges, we propose Workify, a modular and intelligent web platform that centralizes all the services necessary for effective, transparent and professional freelance collaboration. The main objectives of Workify are as follows :

- Centralize freelance management : offer a single ecosystem bringing together project publication, contract management, communication, evaluations and invoicing within one single platform.
- Integrate artificial intelligence : leverage Machine Learning for freelancer-project matching, avatar generation, feedback sentiment analysis and subscription pack recommendation.
- Guarantee security and trust : implement JWT authentication with role management (RBAC), automatic fraud detection and certificate verification by QR code.
- Foster professional development : offer online training programs with automatic generation of PDF certificates verifiable by third parties.
- Ensure scalability : design a modular architecture allowing easy addition of new modules and features.

4. Analysis and Choice of the Working Method

4.1 Definition of the Scrum Method

We chose the Agile Scrum approach to design and develop our Workify platform. This methodology is well suited to the evolving requirements of the project, which perfectly matches the modular and incremental nature of our solution.

Scrum is an agile project management framework that helps teams structure and manage their work according to a set of values, principles and practices. It uses an iterative and incremental development model with short cycles called sprints. Scrum defines three main roles :

- Product Owner : responsible for defining and prioritizing the product backlog, and for maximizing the value delivered at each sprint.
- Scrum Master : ensures the proper application of the methodology and is responsible for removing obstacles that may hinder the team's progress.
- Development Team : composed of five members, responsible for the design, development and testing of features at each sprint.



Figure 1 : Agile Scrum methodology

4.2 Choice of Methodology

The choice of the Scrum method for our project is justified by several reasons :

- The modular nature of Workify with its ten functional modules is perfectly suited to iterative and incremental development.
- The project requirements are likely to evolve throughout the sprints based on the supervisor's feedback and intermediate evaluations.
- Scrum provides better visibility of the deliverable product and promotes continuous communication within the team.
- It also allows quality issues to be detected early and corrective actions to be taken without significant penalties on deadlines.

Our project was therefore divided into four sprints spread over fourteen weeks, each giving rise to a review and an evaluation at the end of the iteration.

Conclusion

The preliminary study is an essential step in the realization of our Workify platform. It allowed us to analyze, evaluate and criticize existing solutions while clearly identifying the needs of the different actors in the freelancing market. The study of existing solutions enabled us to prepare a solid design and define the improvements we will bring through our solution. This first part constitutes the necessary foundation to approach the design of our project, which is the subject of the following chapter.

Chapter 2 : Requirements Analysis and Specification

Introduction

In this chapter, we address the requirements analysis phase. We begin by identifying the actors involved and distinguishing the functional and non-functional requirements that our solution must meet. We then establish the definition of our product backlog and plan the sprints. Finally, we translate these requirements into use case diagrams in order to obtain a precise vision of the project's requirements.

This part is necessary to approach the design of our project, which is the subject of the following chapter.

1. Identification of Actors

The starting point of any application analysis is the identification of its actors. An analysis of the interaction between the system and its external environment reveals the following main actors :

1.1 Administrator

This actor occupies the highest position in the system hierarchy and manages the entire platform. His main role consists of :

- Managing and moderating all user accounts (activation, suspension, deletion)
- Assigning roles and managing role-based access control (RBAC)
- Supervising all financial transactions and subscriptions
- Managing support tickets and resolving disputes between clients and freelancers
- Creating and managing professional events and sponsors
- Accessing global platform statistics and dashboards
- Managing training programs and validating certifications

1.2 Freelancer

This actor is an independent professional who offers his services on the platform. His main role consists of :

- Creating and managing his professional profile with skills and portfolio
- Browsing available projects and submitting offers
- Receiving AI-powered project recommendations based on his skills
- Managing his tasks and planning through the integrated calendar
- Communicating with clients via the internal messaging system

- Receiving and responding to client feedback
- Applying for internship opportunities published by partners
- Accessing training programs and obtaining certified PDF certificates
- Subscribing to packs and managing his subscription

1.3 Client (Project Owner)

This actor is a company or individual looking for freelance talent to carry out projects. His main role consists of :

- Publishing projects with detailed descriptions, budgets and required skills
- Receiving AI-powered freelancer recommendations
- Comparing offers and selecting the most suitable freelancer
- Evaluating freelancers at the end of each project
- Creating support tickets to report any issues
- Communicating with freelancers via the internal messaging system

1.4 Partner

This actor is a company or organization that collaborates with the platform. His main role consists of :

- Publishing internship and job opportunities
- Managing applications submitted by freelancers
- Creating and managing professional training programs
- Issuing and validating certificates linked to completed training

1.5 Service Desk

This actor is responsible for customer support management. His main role consists of :

- Receiving and processing support tickets created by clients
- Assigning tickets to the appropriate agents based on category and priority
- Responding to tickets and updating their status
- Closing resolved tickets and archiving them

2. Functional Requirements

The functional requirements represent the main features that our application must satisfy for its users. Below, we enumerate all the system functionalities :

- User and Access Management : allow users to register, authenticate and manage their profiles; implement RBAC with four roles; enable secure password reset; implement JWT with refresh token; allow AI avatar generation from a real photo.

- **Project and Offer Management** : allow clients to create, edit and delete projects; allow freelancers to submit and cancel offers; implement AI-powered matching; manage the project lifecycle (Open → In Progress → Completed → Closed); automatically mark projects as 'At Risk' in case of inactivity.
- **Task and Planning Management** : allow freelancers to create and track tasks with priorities and deadlines; provide calendar view (day/week/month); send automated deadline reminders.
- **Internal Communication** : allow users to send and manage messages; implement auto-archive and dynamic conversation priority; automatically detect inappropriate content using AI; generate smart conversation summaries.
- **Feedback and Evaluation Management** : allow clients to submit ratings after project completion; allow freelancers to respond; implement fraud detection and AI sentiment analysis (positive / neutral / negative).
- **Event and Sponsor Management** : allow administrators to create events (webinars, conferences, workshops); manage speakers and sponsors; integrate Google Calendar; send notifications via Twilio API.
- **Opportunity and Application Management** : allow partners to publish internship opportunities; allow freelancers to apply; allow partners to accept or reject applications; automatically close filled opportunities.
- **Ticket and Support Management** : allow clients to create support tickets; allow service desk agents to assign, respond to and close tickets; implement auto-assignment and prioritization.
- **Training and Certification Management** : allow partners to create training programs with Google Meet links; automatically generate PDF certificates; allow QR code verification; display certificates on freelancer profiles with public/private visibility.
- **Pack and Subscription Management** : allow administrators to manage subscription packs; allow users to subscribe with online payment; generate automatic PDF invoices; implement AI-powered pack recommendation.

3. Non-Functional Requirements

Non-functional requirements are essential to guarantee the software quality of our system. Our application must imperatively meet the following requirements :

- **Performance** : ensure optimal responsiveness of the platform with a response time of less than 2 seconds for 95% of requests.
- **Security** : implement JWT authentication, BCrypt password encryption, RBAC and protection against unauthorized access.

- Ergonomics and Usability : offer simple and intuitive interfaces, compatible with modern browsers (Chrome, Firefox, Edge, Safari).
- Scalability : design an extensible and modular architecture to allow the addition of new features and modules.
- Reliability : ensure 99.9% platform availability with automated status handling and data consistency checks.
- Real-time : manage messaging, notifications and chat functionalities instantaneously to offer a smooth user experience.
- Maintainability : ensure code coverage greater than 80% through unit and integration tests, with complete Swagger API documentation.

4. Product Backlog

The Product Backlog is the central element of Scrum, defining the functionalities to be implemented in the project. Each sprint results in the delivery of a functional partial product called an increment. The product backlog of our project is as follows :

ID	As a	User Story	Tasks	Priority
1	Admin	Manage users	Register / authenticate / edit / delete accounts	1
2	Admin	Manage roles	Assign roles, activate / suspend accounts (RBAC)	1
3	All	Authentication	Login, logout, password reset, token management	1
4	All	AI Avatar	Generate AI avatar from real photo, anonymization	2
5	Client	Manage projects	Create / edit / delete / change project status	1
6	Client	Select freelancer	Compare offers, choose freelancer	1
7	Freelancer	Manage offers	Submit / edit / cancel offer	1
8	All	AI Matching	Recommend best freelancer for project	2
9	All	Project lifecycle	Status transitions (open→in progress→completed→closed)	1
10	Client	Start conversation	Create new conversation with freelancer	1

ID	As a	User Story	Tasks	Priority
11	Freelancer	Reply to conversation	Send messages	1
12	All	Moderation	Auto-archive, bad words filter	2
13	All	AI summary	Generate smart conversation summary	3
14	Client	Submit feedback	Rate project and write comment	1
15	Freelancer	Respond to feedback	Write response to client feedback	1
16	All	Fraud detection	Detect duplicate / suspicious feedback	3
17	All	AI sentiment	Detect tone (positive / neutral / negative)	3
18	Admin	Manage events	Create / edit / delete event	2
19	Admin	Manage sponsors	CRUD sponsor details	2
20	Partner	Publish opportunity	CRUD internship opportunity	1
21	Freelancer	Apply to opportunity	Submit application	1
22	Client	Create ticket	CRUD support ticket	1
23	Service Desk	Manage ticket	Respond, update status, close ticket	1
24	Partner	Manage training	CRUD training programs with domain and duration	2
25	Freelancer	Obtain certificate	Link training to certificate, PDF download	2
26	Admin	Manage packs	CRUD subscription pack	1
27	User	Manage subscription	Subscribe / cancel / update status	1
28	User	Payment & billing	Online payment, multiple methods, PDF invoice	1
29	All	AI recommendation	Suggest best pack (upgrade / downgrade)	3
30	Freelancer	Manage tasks	CRUD tasks, change status (To Do/In Progress/Done)	1
31	Freelancer	Calendar view	Day / week / month navigation	2

Table 5 : Product Backlog

5. Sprint Planning

Once the product backlog was finalized, we organized a planning meeting in order to establish the sprint backlog. To structure our project, we decided to divide it into four sprints, as specified in the following table :

Sprint	Objectives	User Stories	Duration
Sprint 0	Requirements analysis, UML modeling, architecture design	Actors, use case diagrams, class diagram, backlog	Week 1 → Week 2
Sprint 1	User management, project & offer, AI matching (v1)	US 1, 2, 3, 4, 5, 6, 7, 8, 9	Week 3 → Week 6
Sprint 2	Communication, feedback, events, opportunities, tickets, tasks	US 10–23, 30, 31	Week 7 → Week 10
Sprint 3	Training, certification, subscriptions, DevOps	US 24–29 + Docker, Jenkins, Kubernetes, SonarQube	Week 11 → Week 14

Table 6 : Sprint planning

6. Use Case Identification

The use case diagram is a key element of UML, offering a graphical representation of the interactions between users and the software system. It is useful for understanding the functional requirements of the system and for communicating with stakeholders.

6.1 Global Use Case Diagram

The global use case diagram below illustrates the interactions between the five main actors — Administrator, Freelancer, Client, Partner and Service Desk — and all the system functionalities :



Figure 2 : Global use case diagram

6.2 Class Diagram

The general class diagram provides an overview of the essential elements of our application. It highlights the main classes as well as their methods, properties and the relationships between them :

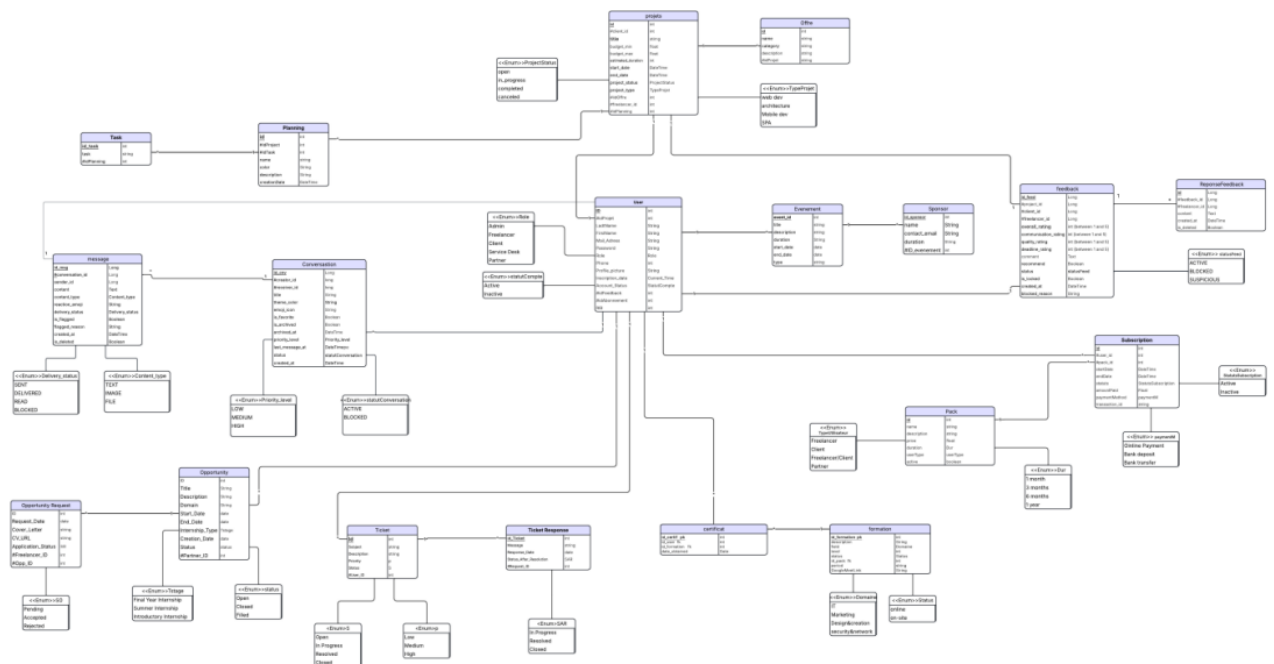


Figure 3 : Global class diagram

Conclusion

In the first part of this chapter, we explored in depth the requirements of our application, both functional and non-functional. We identified the five main actors of the Workify platform and detailed the features each actor needs to interact with the system. The second part was devoted to the product backlog and sprint planning. We also examined the global use case and class diagrams. In the next chapter, we move on to the design and implementation phase, detailing the realization of each sprint.

Chapter 3 : Conceptual Study and Implementation

Introduction

This chapter is devoted to the design and implementation phase of the Workify platform. It presents the realization of each sprint following the Agile Scrum methodology. For each sprint, we detail the sprint backlog, the conceptual design through UML diagrams, the implementation of the developed features and the tests carried out to validate the results. This chapter also covers the versioning strategy adopted using GitHub and the complete DevOps pipeline deployed in Sprint 3.

1. Sprint 0 : Analysis and Specification

After finalizing the project context and requirements, we begin with Sprint 0, which constitutes the foundation of the entire project. This sprint is dedicated to analysis, modeling and architectural design before any line of code is written.

1.1 Sprint Backlog

Epic	User Story	Detailed Description
Sprint 0	Define system actors	Identify all actors and their interactions with the system. Five main actors are identified : Administrator, Freelancer, Client, Partner and Service Desk.
Sprint 0	Specify functional requirements	List all functional requirements per module with their priorities. Ten modules and 31 user stories are defined.
Sprint 0	Produce global use case diagram	Model all actor-system interactions in UML. Each actor is associated with his specific set of use cases.
Sprint 0	Design class diagram	Design all entities, attributes, relationships and cardinalities. Sixteen main entities are identified with their associations.
Sprint 0	Define global architecture	Choose the technology stack and design the three-tier system architecture (Angular / Spring Boot / MySQL).
Sprint 0	Establish product backlog	Prioritize all 31 user stories and assign them to the four sprints according to their priority and complexity.

Table 7 : Sprint 0 backlog

1.4 Global Architecture

The Workify platform is built on a three-tier architecture that clearly separates the presentation layer (Angular 17), the business logic layer (Spring Boot 3.x) and the data layer (MySQL). This architecture ensures a clean separation of concerns, facilitating maintenance, testing and future evolution of the platform.

Layer	Technology	Role
Presentation Layer	Angular 17	Single Page Application (SPA), user interface
Business Logic Layer	Spring Boot 3.x	REST API, business rules, security management
Data Layer	MySQL 8.0	Relational data persistence
Real-time Layer	WebSocket (STOMP)	Instant messaging and live notifications
AI Layer	Python / ML Models	Matching, avatar generation, sentiment analysis
Containerization	Docker	Application packaging and environment isolation
Orchestration	Kubernetes	Deployment, scaling and high availability
CI/CD	Jenkins / GitHub Actions	Continuous integration and delivery pipeline
Code Quality	SonarQube	Static analysis and quality gate enforcement

Table 8 : Technology stack

2. Sprint 1 : User Management, Project & Offer, AI Matching

After completing Sprint 0, we now turn to the planning and execution of Sprint 1, which covers the core modules of the Workify platform. This sprint was estimated at three to four weeks.

2.1 Sprint Backlog

Epic	User Story	Detailed Description
Sprint 1.1 — User Management	Manage user accounts	After registration, the administrator can view, activate, suspend and delete user accounts. Role-based access control (RBAC) is enforced for all platform actions. Four roles are supported : Admin, Freelancer, Client and Partner.

Epic	User Story	Detailed Description
Sprint 1.1 — User Management	Authentication & security	Users authenticate via email and password. The system generates a JWT access token upon successful login. A refresh token mechanism ensures seamless session continuity without requiring re-authentication.
Sprint 1.1 — User Management	AI Avatar generation	The freelancer can upload a real photo. The system uses a pre-trained AI model to automatically generate an anonymized avatar. The user can toggle between displaying the real photo or the AI-generated avatar on his public profile.
Sprint 1.2 — Project & Offer	Manage projects	After authentication, the client can create a project by providing title, description, budget, required skills and deadline. He can also edit, delete and update the project status at any stage of its lifecycle (Open → In Progress → Completed → Closed).
Sprint 1.2 — Project & Offer	Manage offers	After authentication, the freelancer can browse open projects and submit an offer with his proposed hourly rate, timeline and relevant skills. He can edit or cancel his offer at any time before it is accepted by the client.
Sprint 1.3 — AI Matching	AI Matching algorithm	The system automatically analyzes the project requirements and compares them with all available freelancer profiles using a weighted multi-criteria scoring algorithm. The client receives a ranked list of the most suitable recommended freelancers on his dashboard.

Table 9 : Sprint 1 backlog

Use Case Description : Authenticate**Title :** Authentication**Goal :** Allow users to securely access the platform according to their role**Actor :** Admin, Freelancer, Client, Partner, Service Desk**Precondition :** The user has a registered account on the platform**Postcondition :** The user is authenticated and redirected to his role-specific dashboard**Main Scenario :**

- The user accesses the login page of the Workify platform
- The user enters his email address and password
- The system validates the credentials against the database
- The system generates a JWT access token and a refresh token
- The user is redirected to his dashboard according to his assigned role

Alternative Scenario :

- If the credentials are incorrect, the system displays an error message and allows the user to retry
- After 5 failed attempts, the account is temporarily locked for security purposes

Implementation — User Management

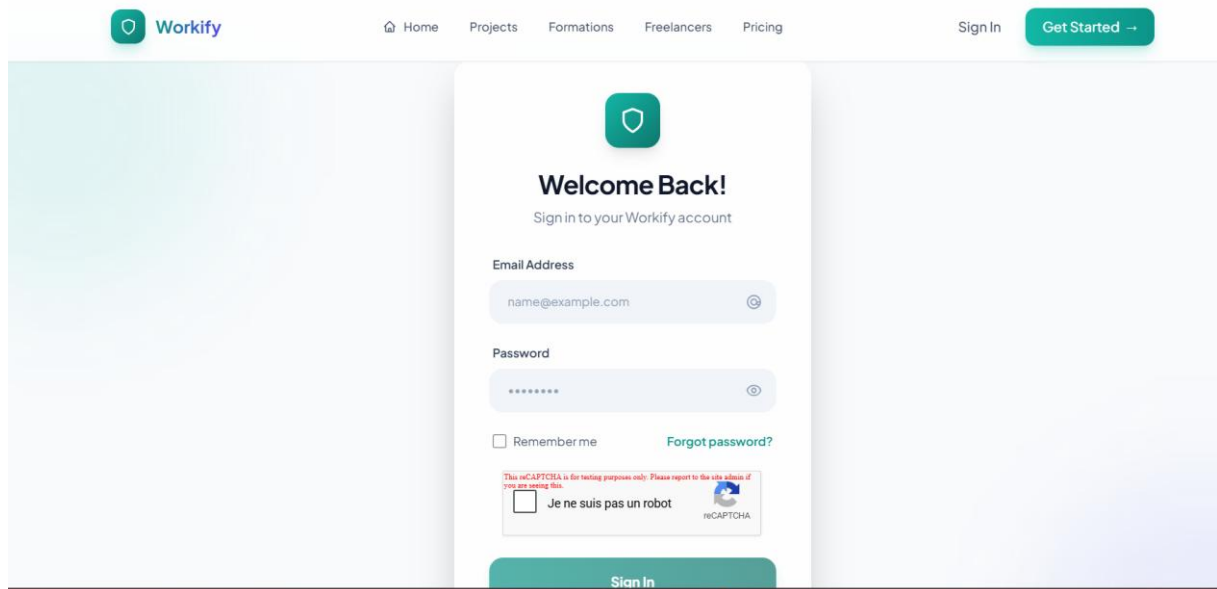


Figure 4 : login interface

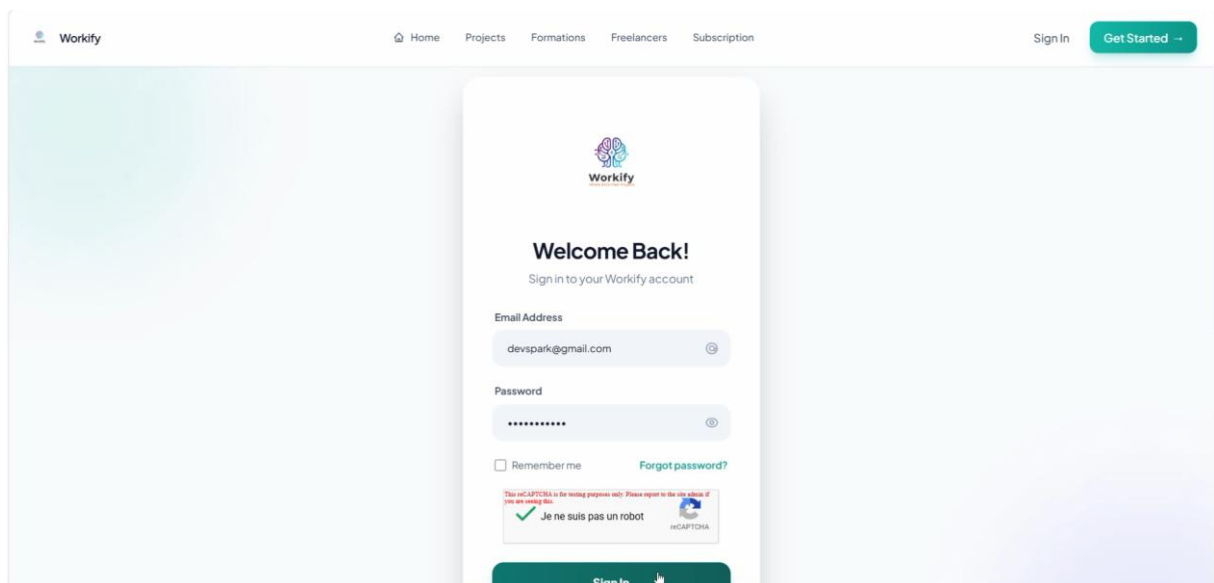


Figure 5 : User registration

2.5 Sequence— Publish a Project

Title : Publish a Project

Goal : Allow the client to publish a project visible to all freelancers

Actor : Client (authenticated)

Precondition : The client is authenticated on the platform

Postcondition : The project is published with 'Open' status and freelancers can submit offers

Main Scenario :

- The client clicks on 'Create Project' in his dashboard
- The client fills in the project details : title, description, budget, required skills and deadline
- The system validates the entered information
- The project is saved in the database and published with 'Open' status

Alternative Scenario :

- If mandatory fields are missing, the system displays validation error messages
- The client corrects the errors and resubmits the form

Implementation — Project & Offer Management

Figure

Figure6: Project list and creation interface

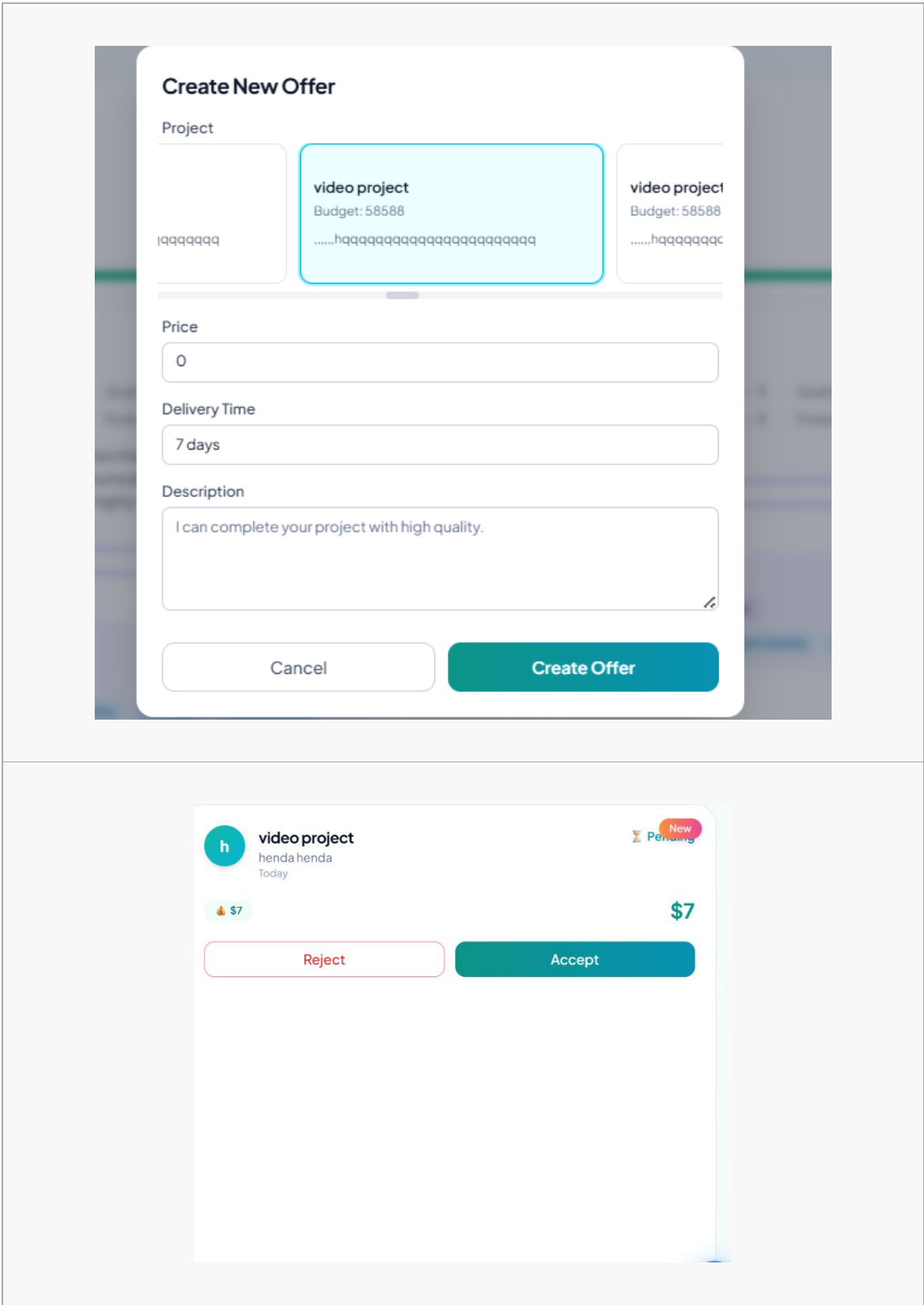


Figure7: Offer submission and comparison interface

2 AI Matching — Criteria and Weights

Criterion	Weight	Description
Skills match	40%	TF-IDF comparison of required vs. possessed skills
Historical success rate	20%	Score based on previously completed and validated projects
Average rating	20%	Global evaluation score of the freelancer on the platform
Availability	10%	Current workload and calendar availability of the freelancer
Daily rate	10%	Adequacy of the freelancer's rate with the project budget

Table 10 : AI Matching criteria and weights

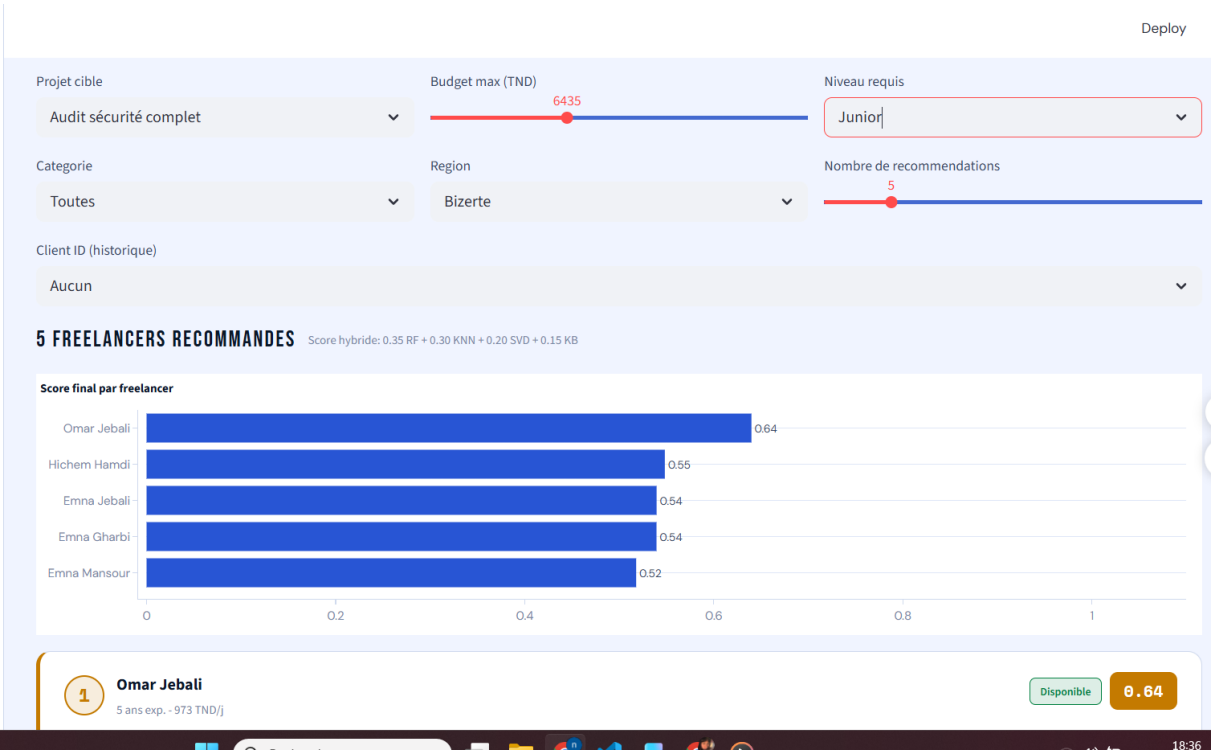


Figure8: AI Matching results — Recommended freelancers interface

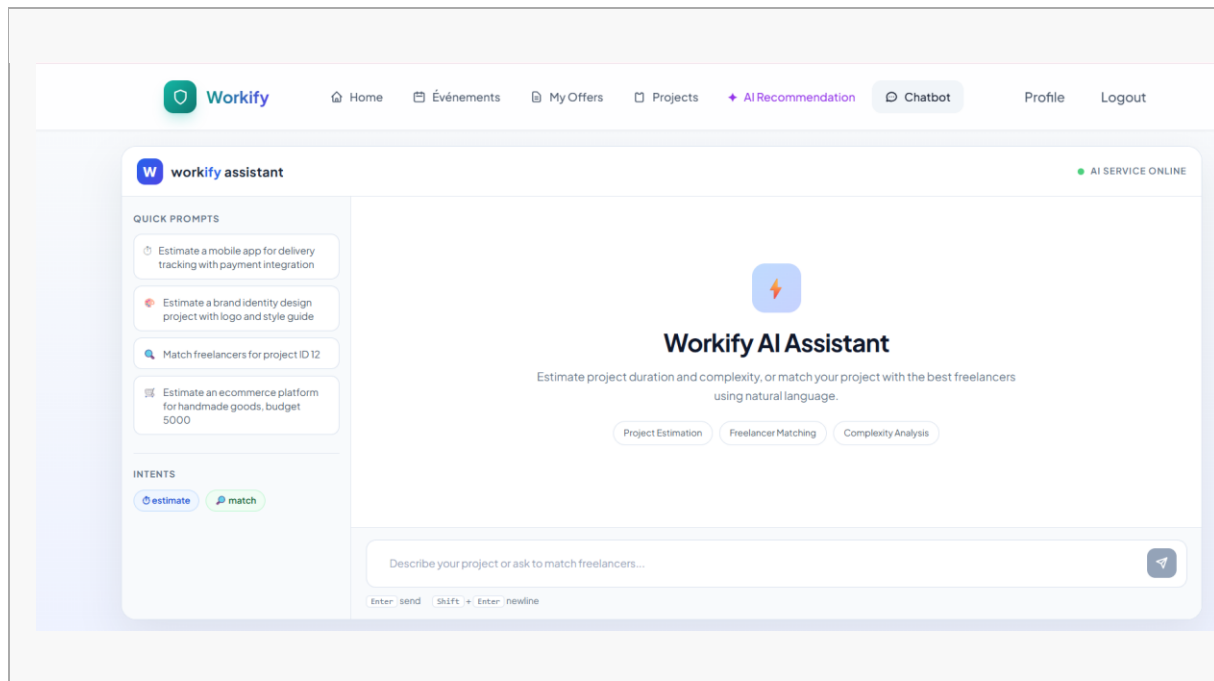


Figure9: AI Matching results — Recommended freelancers interface

3. Sprint 2 : Communication, Feedback, Events, Opportunities, Tickets & Tasks

After completing Sprint 1, we now turn to the planning and execution of Sprint 2. This sprint covers six functional modules and was estimated at three to four weeks.

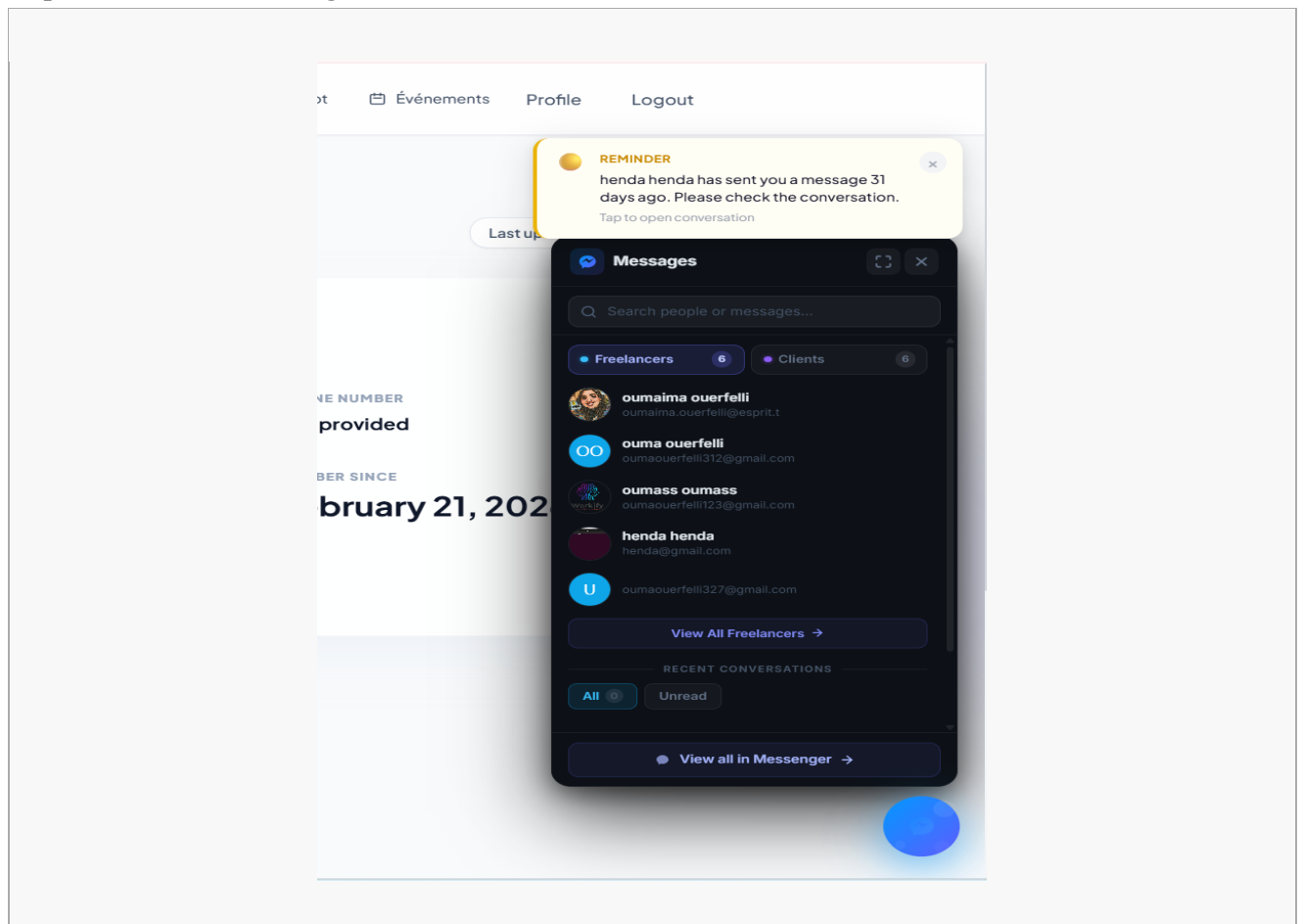
3.1 Sprint Backlog

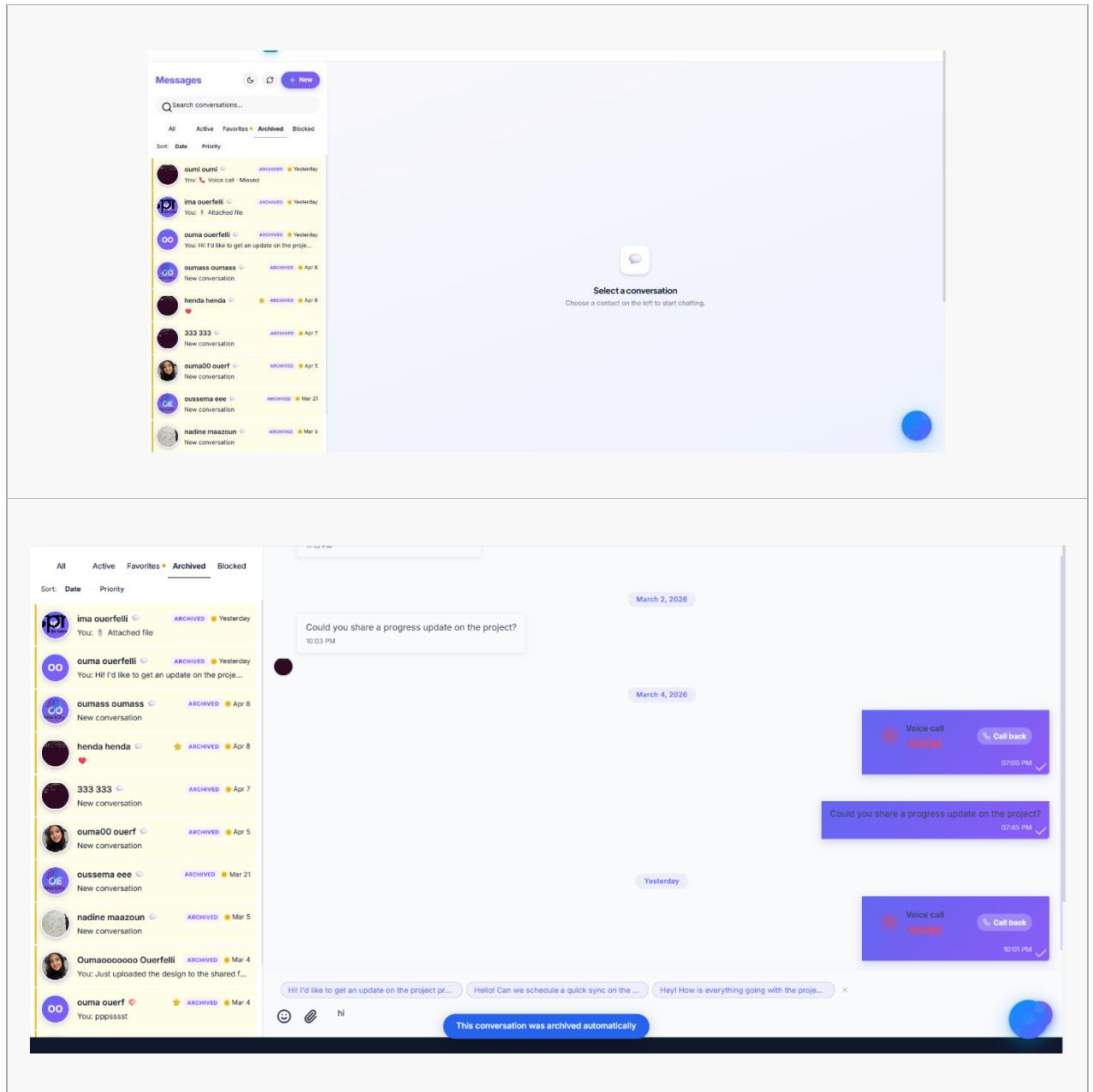
Epic	User Story	Detailed Description
Sprint 2.1 — Messaging	Manage conversations and messages	Users can create, send, edit and delete messages in real time via WebSocket. A dynamic priority system automatically manages conversation visibility. Conversations inactive for more than 10-15 minutes are automatically archived.
Sprint 2.1 — Messaging	AI content moderation	The system automatically detects inappropriate content. If a bad word is detected, the message is blocked and the sender receives a warning email. After 3 violations, the user is temporarily blocked from sending new messages.
Sprint 2.2 — Feedback	Submit and manage feedback	After project completion, the client can submit a rating (1-5 stars) with a comment. Feedback can only be edited within 10 minutes of submission. The freelancer can respond to received feedback.
Sprint 2.2 — Feedback	Fraud detection & AI sentiment	The system detects suspicious feedback behavior (multiple feedbacks submitted rapidly for the same freelancer). An AI sentiment analysis model classifies the tone of each feedback as positive, neutral or negative.

Epic	User Story	Detailed Description
Sprint 2.3 — Events	Manage events and sponsors	Administrators can create professional events (webinars, conferences, workshops). Sponsors can propose services and resources. Google Calendar integration automatically sends reminders to registered participants.
Sprint 2.4 — Opportunities	Manage internship opportunities	Partners can publish internship opportunities with title, description, required skills and deadline. Freelancers can apply by submitting their resume. Opportunities are automatically closed once they are filled.
Sprint 2.5 — Tickets	Manage support tickets	Clients can create support tickets to report issues. Tickets are automatically assigned to service desk agents based on their category and priority level. Status is tracked from open to resolved to closed.
Sprint 2.6 — Tasks	Manage tasks and planning	Freelancers can create tasks with priorities and deadlines. A calendar view (day/week/month) provides a clear overview of the workload. Automated reminders are sent before approaching deadlines.

Table 11 : Sprint 2 backlog

Implementation — Message & Conversation





Use Case Description : Submit Feedback

Title : Submit Feedback

Goal : Allow the client to evaluate the freelancer's work after project completion

Actor : Client (authenticated)

Precondition : The project is completed and the contract is closed

Postcondition : The feedback is saved and the freelancer receives a notification

Main Scenario :

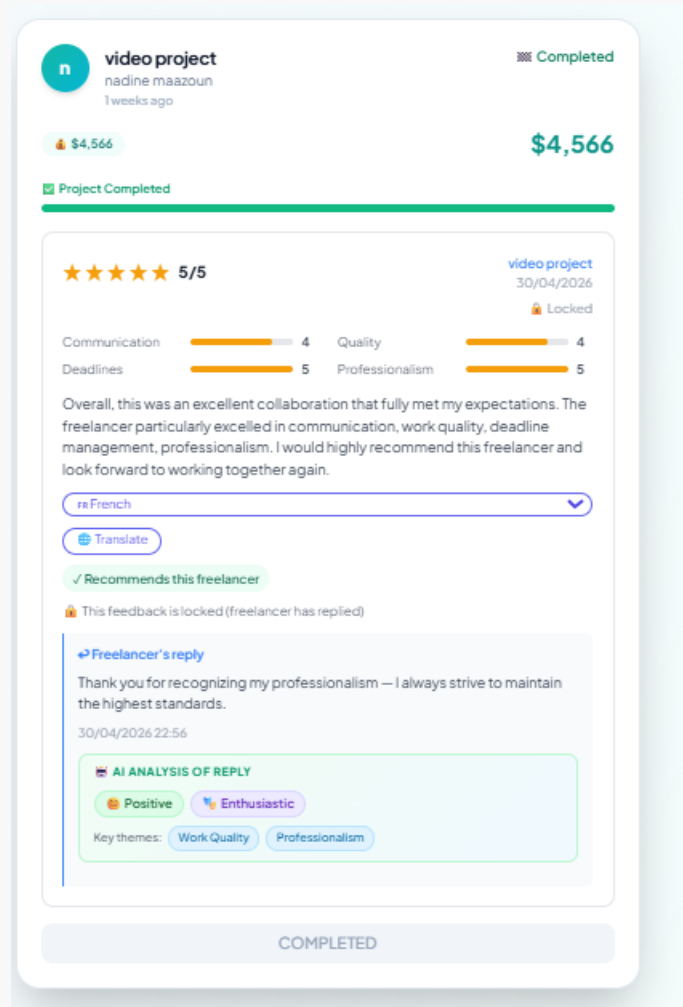
- The client navigates to the completed project page
- The client selects a rating from 1 to 5 stars and writes a comment

- The system verifies for fraudulent behavior before saving the feedback
- The AI sentiment analysis model classifies the tone of the feedback
- The feedback is published and the freelancer receives an alert notification

Alternative Scenario :

- If suspicious behavior is detected, the system flags the feedback for administrator review
- If inappropriate language is detected in the comment, the feedback is rejected

Implementation — Feedback & Response



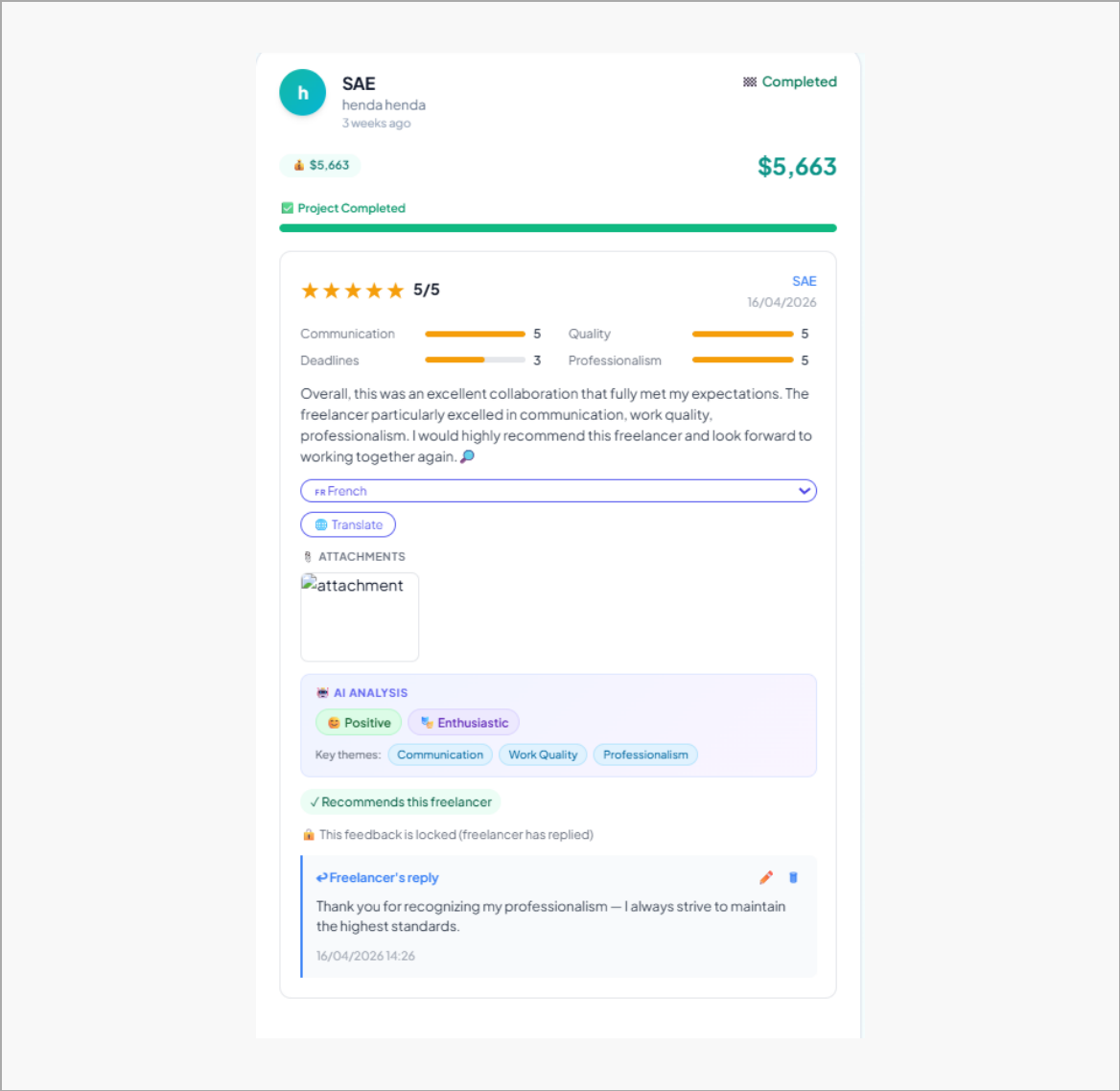


Figure 12 : Feedback submission and list interface

3.6 Implementation — Event & Sponsor

[Home](#) [Backoffice](#) [Profile](#) [Logout](#)

Titre	Catégorie	Date	Lieu	Capacité	Inscrits	Statut	Actions
Conférence Tech 1	TECHNOLOGY	15/01/2025	Tunis, Tunisia	100	0	DRAFT	
Conférence Tech 2	TECHNOLOGY	15/02/2025	Tunis, Tunisia	100	0	DRAFT	
Conférence Tech 3	TECHNOLOGY	15/03/2025	Tunis, Tunisia	100	0	DRAFT	
Conférence Tech 4	TECHNOLOGY	15/04/2025	Tunis, Tunisia	100	0	DRAFT	
Conférence Tech 5	TECHNOLOGY	15/05/2025	Tunis, Tunisia	100	0	DRAFT	
Conf Tech 1	TECHNOLOGY	15/06/2025	Tunis	100	0	DRAFT	
DevOps Summit 2025	TECHNOLOGY	20/07/2025	Sfax, Tunisia	200	0	DRAFT	
Hackathon Web Dev	WEB_DEVELOPMENT	10/08/2025	Sousse, Tunisia	60	0	DRAFT	
Cybersecurity Workshop	CYBERSECURITY	05/09/2025	Tunis, Tunisia	80	0	DRAFT	
Data Science Meetup	DATA_SCIENCE	15/10/2025	Ariana, Tunisia	120	0	DRAFT	

Figure 13 : Event creation and management interface

3.7 Implementation — Opportunity & Request

[Home](#) [Dashboard](#) [Opportunities](#) [Manage Opportunities](#) [Mes Invitations](#)

[partner partner](#)

Partner Opportunities

Manage your opportunities, applications, and rankings.

Total Opportunities Posted

3

Total Applications Received

6

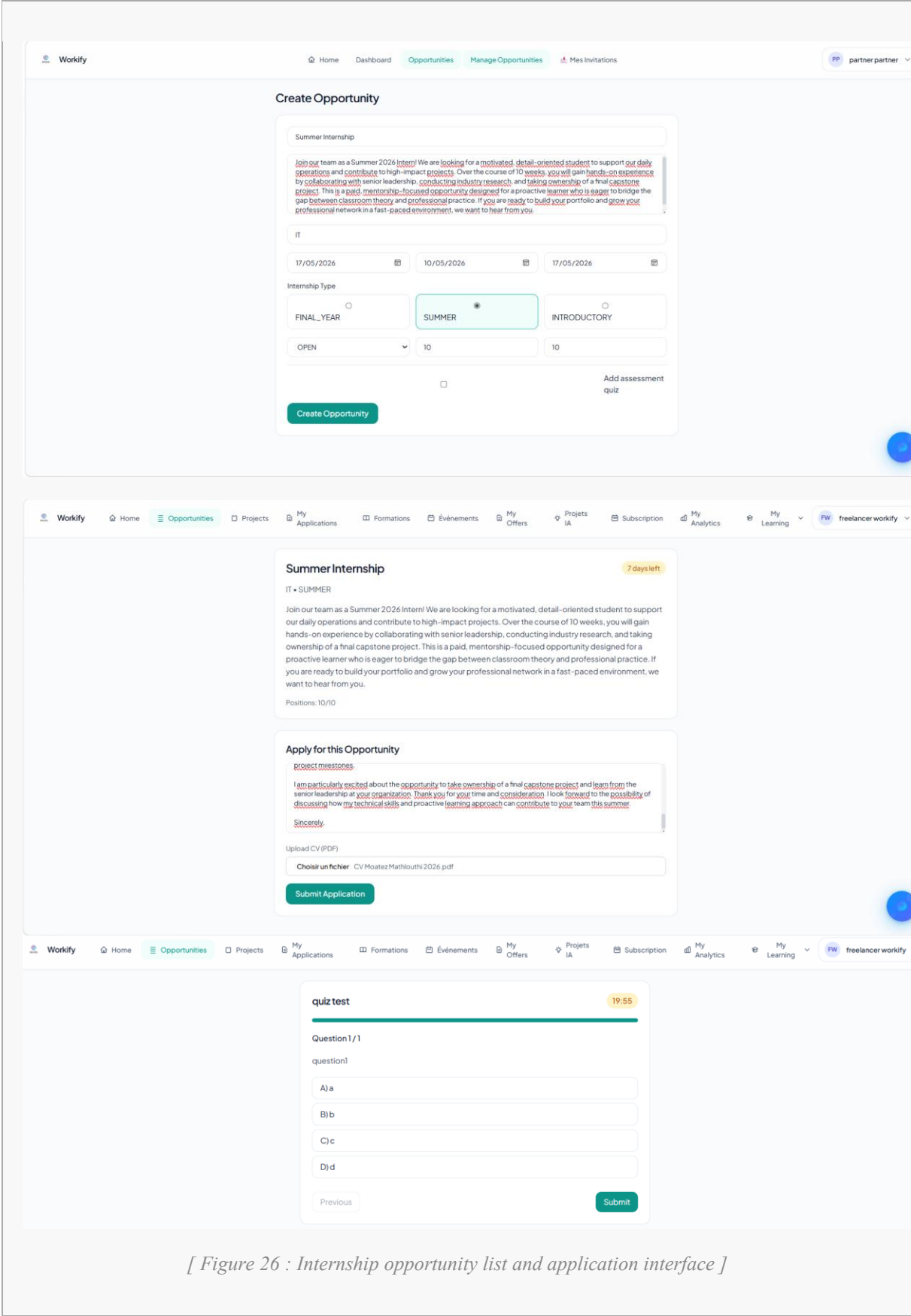
Acceptance Rate

50%

Opportunity Status:

OPEN: 3 FILLED: 0 CLOSED: 0

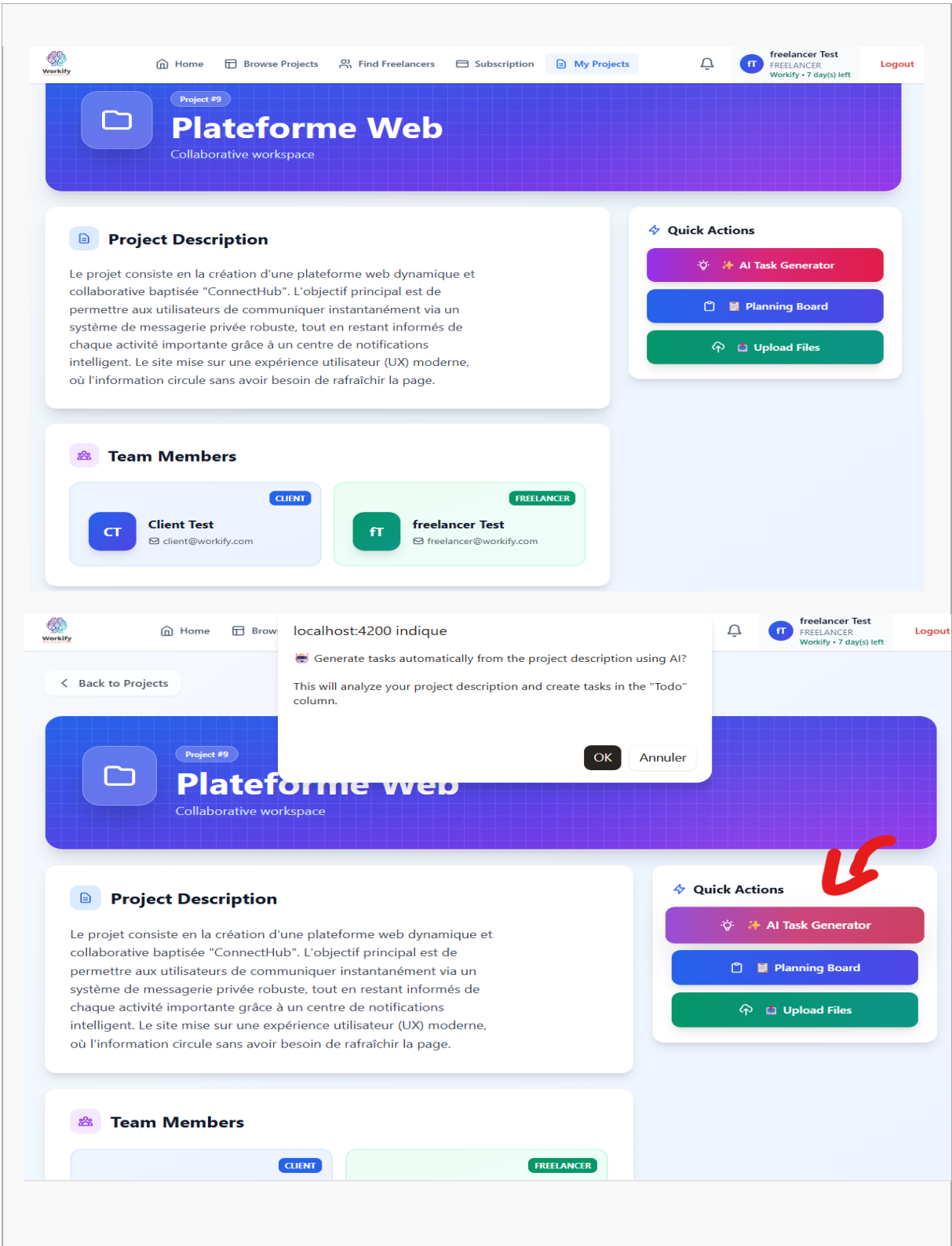
Title	Domain	Internship Type	Status	Positions	Applications	Application Deadline	Actions
testttt	Tech	FINAL_YEAR	OPEN	19/20	2	Apr 26, 2026	Edit Delete View Applications View Rankings
Nour	eli theb	SUMMER	OPEN	9/10	2	Apr 30, 2026	Edit Delete View Applications View Rankings
new oopp	IT	INTRODUCTORY	OPEN	19/20	2	Apr 24, 2026	Edit Delete View Applications View Rankings

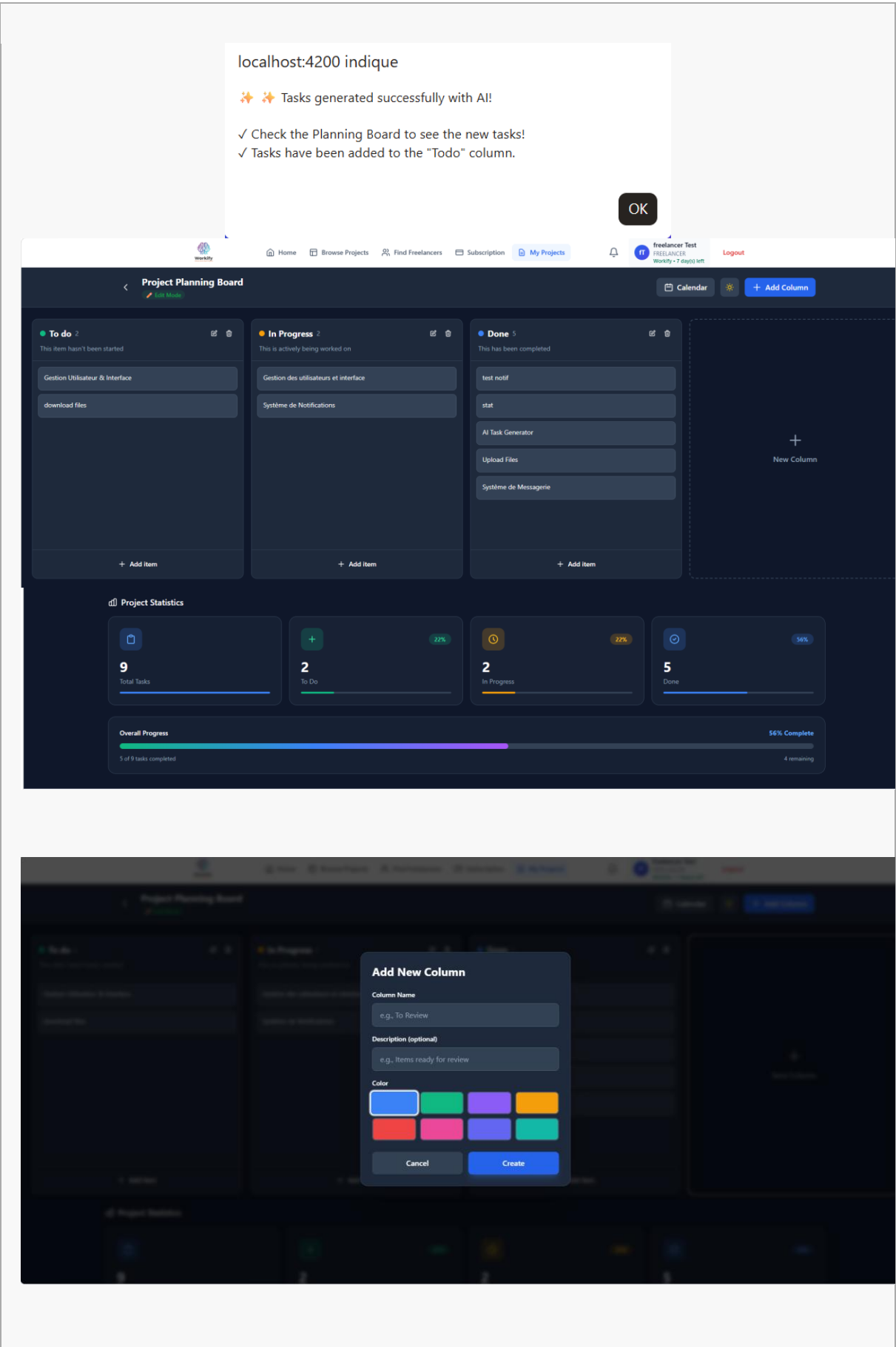


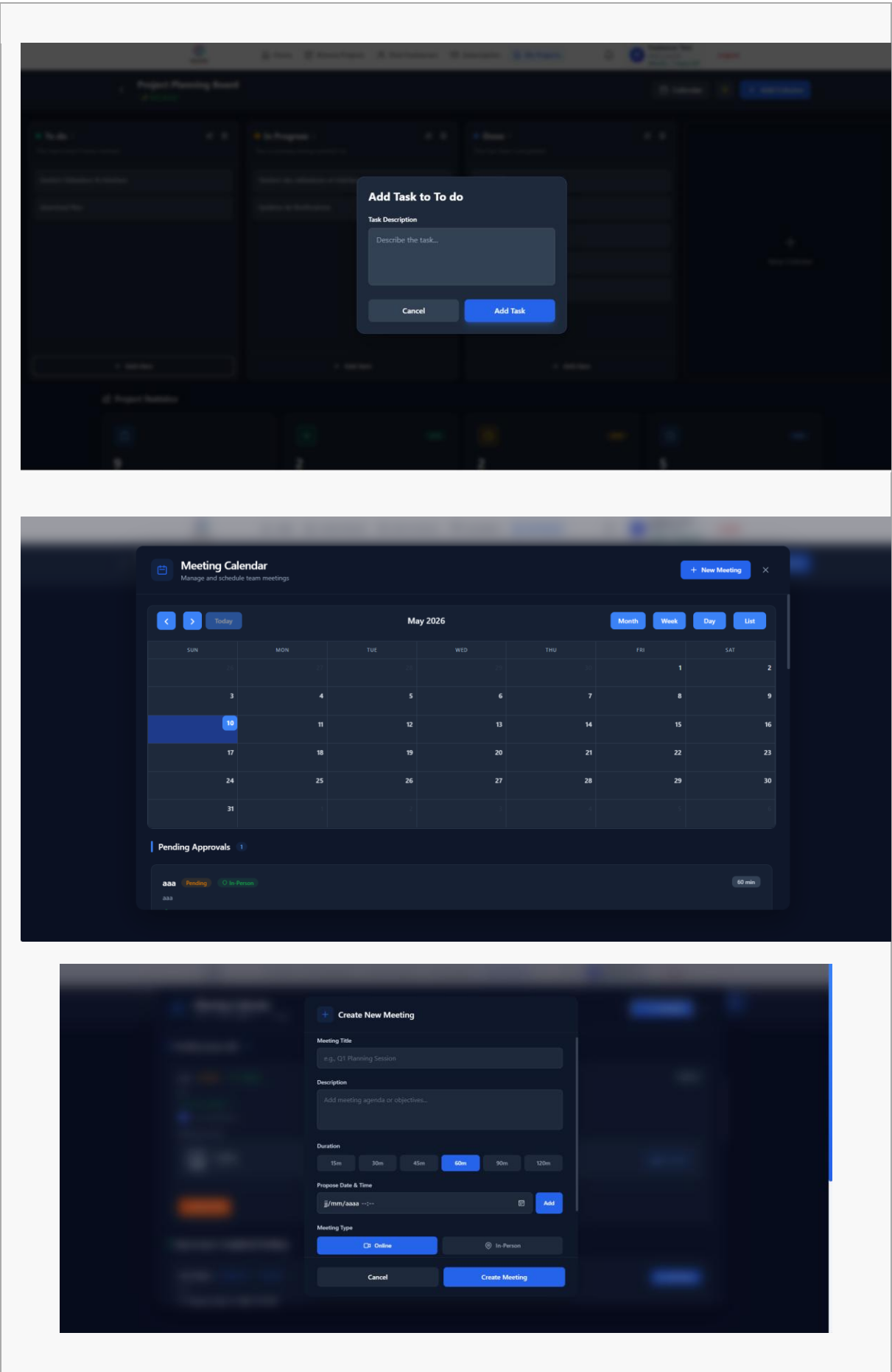
[Figure 26 : Internship opportunity list and application interface]

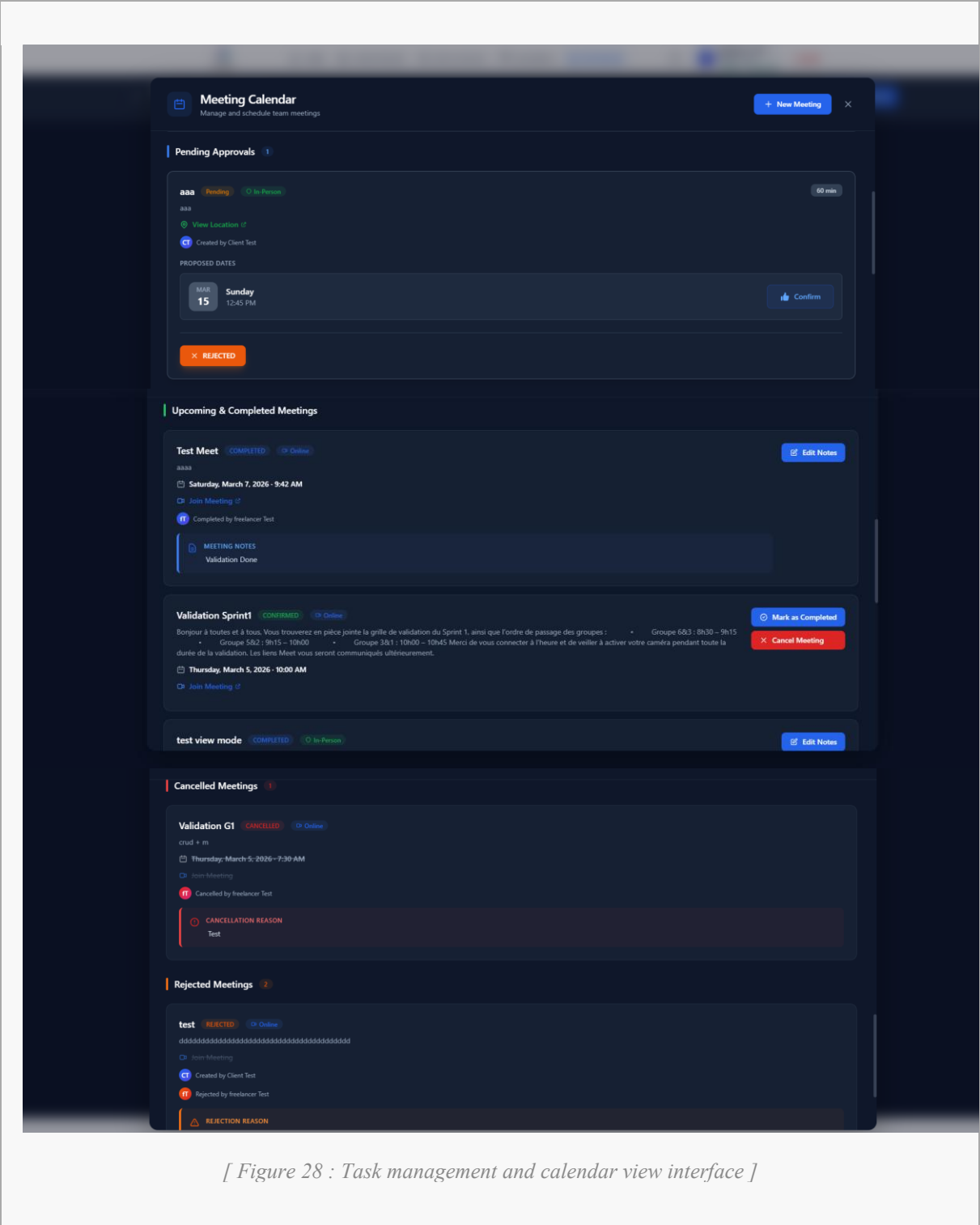
Figure 14 : Internship opportunity list and application interface

3.9 Implementation — Task & Planning









[Figure 28 : Task management and calendar view interface]

Figure 16 : Task management and calendar view interface

4. Sprint 3 : Training, Certification, Subscriptions & DevOps

After finalizing the previous sprints, we now begin the design and execution of the last sprint. This sprint covers the training and certification modules, the subscription and payment system, and the complete DevOps pipeline. It was estimated at three to four weeks.

4.1 Sprint Backlog

Epic	User Story	Detailed Description
Sprint 3.1 — Training	Manage training programs	Partners can create training programs with title, domain, level, description and Google Meet session link. They can assign training to specific freelancers and monitor their completion status (In Progress / Completed).
Sprint 3.1 — Training	Obtain certificate	After completing and validating a training, the system automatically generates a PDF certificate with a unique identifier and a QR code. The freelancer can download the certificate and display it on his profile with public or private visibility settings.
Sprint 3.2 — Subscription	Manage packs and subscriptions	Administrators define subscription packs with different feature levels and pricing. Users can subscribe, cancel or update their subscription. The system tracks active and expired subscriptions and sends expiration notifications.
Sprint 3.2 — Subscription	Payment & invoicing	Users can pay online using multiple methods (bank transfer, deposit, online payment). The system automatically generates a PDF invoice after each successful payment and archives it in the user's account.
Sprint 3.2 — Subscription	AI pack recommendation	The system analyzes the user's activity (project count, revenue generated, usage frequency) and automatically suggests the most suitable subscription pack with intelligent upgrade or downgrade proposals.
Sprint 3.3 — DevOps	Docker containerization	The application is divided into independent Docker containers : frontend (Angular + Nginx), backend (Spring Boot + JRE 17), database (MySQL 8.0), cache (Redis) and quality analysis (SonarQube).
Sprint 3.3 — DevOps	Jenkins CI/CD pipeline	A Jenkins pipeline with 8 automated stages is defined via a Jenkinsfile versioned in the Git repository. It is triggered automatically on each push to the main branches, covering build, tests, quality gate, Docker build and Kubernetes deployment.

Epic	User Story	Detailed Description
Sprint 3.3 — DevOps	Kubernetes deployment	The application is deployed on a Kubernetes cluster with 3 backend replicas and 2 frontend replicas for high availability. HorizontalPodAutoscaler enables automatic scaling based on CPU and memory consumption.

Table 12 : Sprint 3 backlog

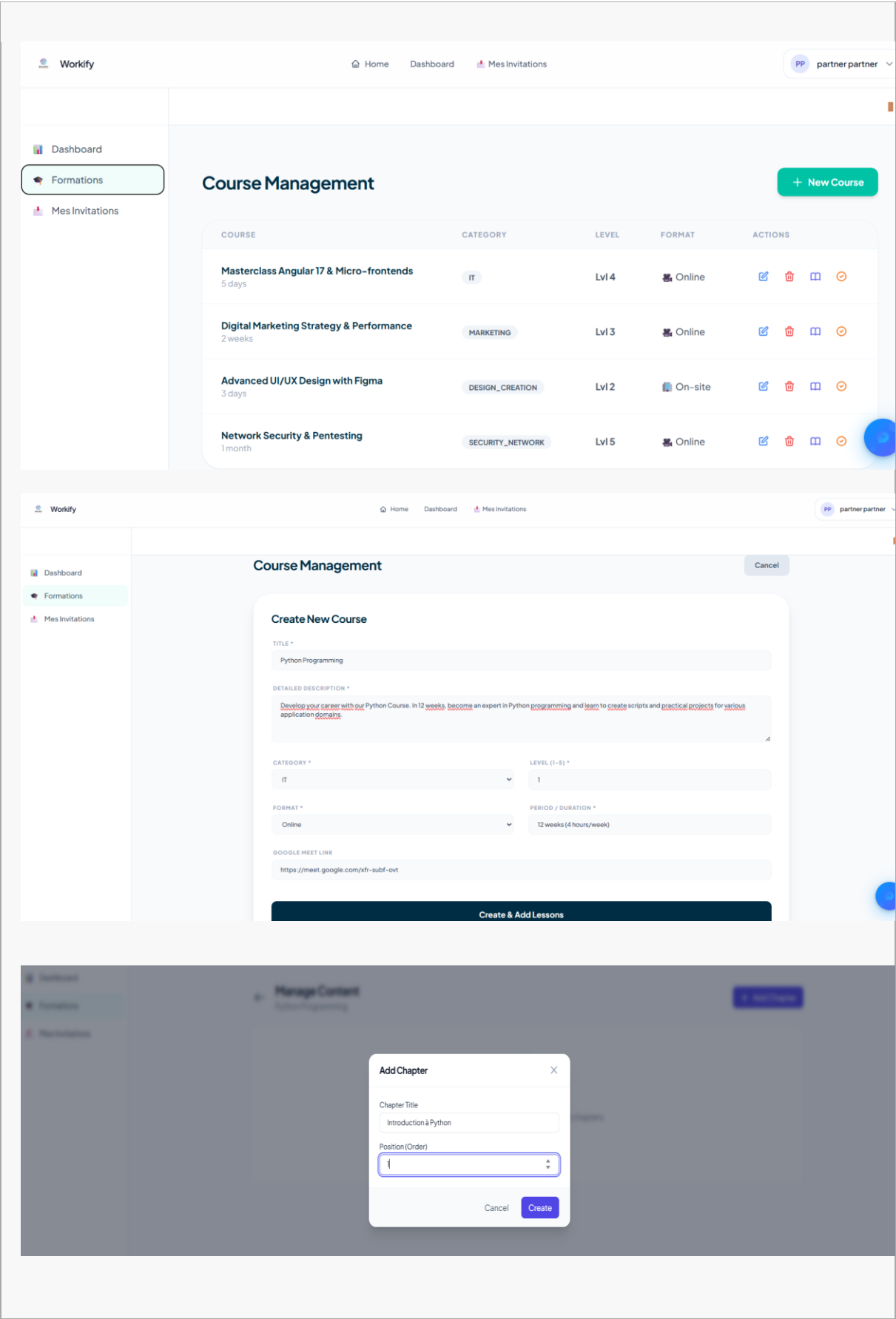
Use Case Description : Generate Certificate**Title :** Generate Certificate**Goal :** Automatically generate a verifiable PDF certificate after training completion**Actor :** System (automatic), Freelancer**Precondition :** The freelancer has completed and the partner has validated the training program**Postcondition :** A unique PDF certificate is generated and associated with the freelancer's profile**Main Scenario :**

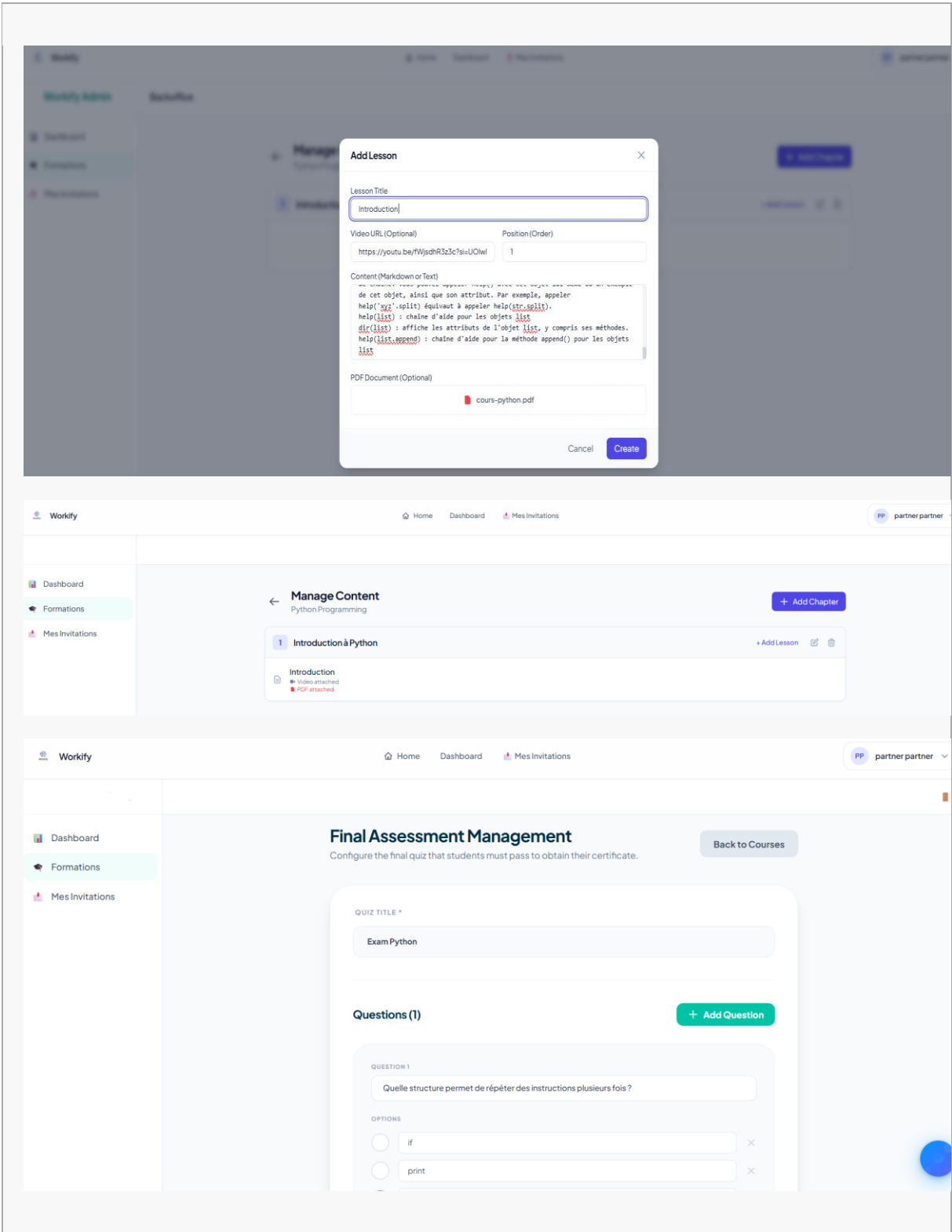
- The partner validates the freelancer's training completion
- The system automatically generates a PDF certificate with a unique identifier and a QR code
- The certificate is saved in the database and linked to the freelancer's profile
- The freelancer receives a notification and can download the PDF certificate
- Third parties can verify the certificate authenticity via the public QR code verification link

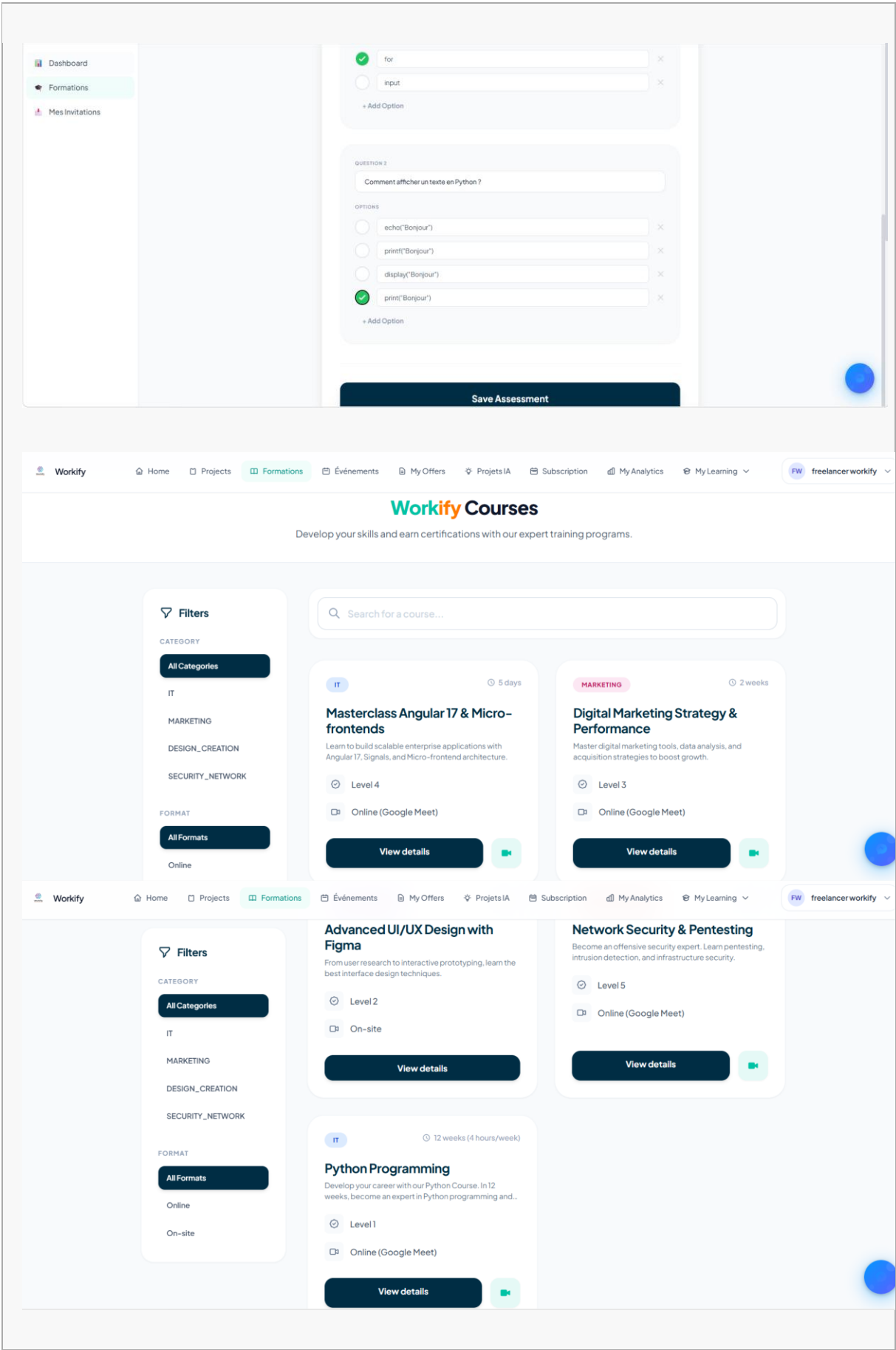
Alternative Scenario :

- If the training is not fully completed, the system prevents certificate generation and notifies the freelancer of the remaining requirements

Implementation — Certificate & Training







Workify

Home

Projects

Formations

Evénements

My Offers

Projets IA

Subscription

My Analytics

My Learning

FWfreelancer workify

← BACK

IT

LEVEL 1

Python Programming

🕒 Duration: 12 weeks (4 hours/week)

📍 Online session

Enroll in this course

Join the session

📄 Certification included

Succeed in this training and receive your official Workify certificate to add to your profile.

🔥 Intensive content

About this course

Develop your career with our Python Course. In 12 weeks, become an expert in Python programming and learn to create scripts and practical projects for various application domains.

What you will learn

✓ Mastery of fundamental concepts in IT

✓ Mastery of fundamental concepts in IT

✓ Mastery of fundamental concepts in IT

✓ Mastery of fundamental concepts in IT

Practical details

FormatOnline

Duration12 weeks (4 hours/week)

Level1

CertificateIncluded

🕒 Need help?

Our advisors are available to answer your questions about this training.

Contact an expert

FOR CLIENTS

FOR FREELANCERS

COMPANY

LEGAL

Workify

Home

Projects

Formations

Evénements

My Offers

Projets IA

Subscription

My Analytics

My Learning

FWfreelancer workify

My Learning Journey

Tracks your progress and continue where you left off.

Explore Courses

IT

Masterclass Angular 17 & Micro-frontends

PROGRESS0%

Lvl 4Enrolled: Apr 15

Continue Learning →

MARKETING

Digital Marketing Strategy & Performance

PROGRESS0%

Lvl 3Enrolled: Apr 15

Continue Learning →

SECURITY_NETWORK

Network Security & Pentesting

PROGRESS0%

Lvl 5Enrolled: Apr 15

Continue Learning →

DESIGN_CREATION

Advanced UI/UX Design with Figma

PROGRESS0%

Lvl 2Enrolled: Apr 15

Continue Learning →

IT

Python Programming

COMPLETED

PROGRESS100%

Lvl 1Enrolled: May 10

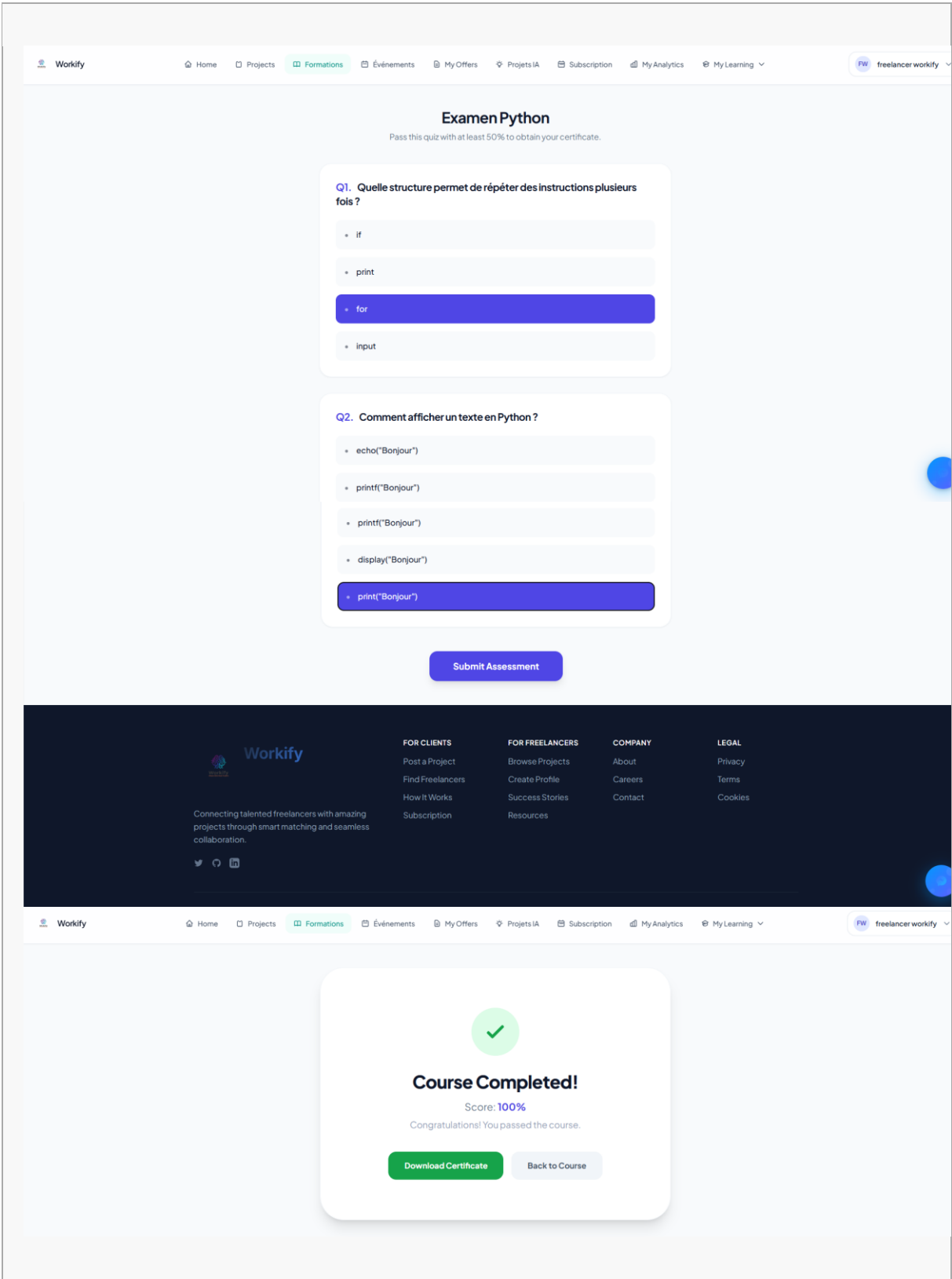
Review Content →

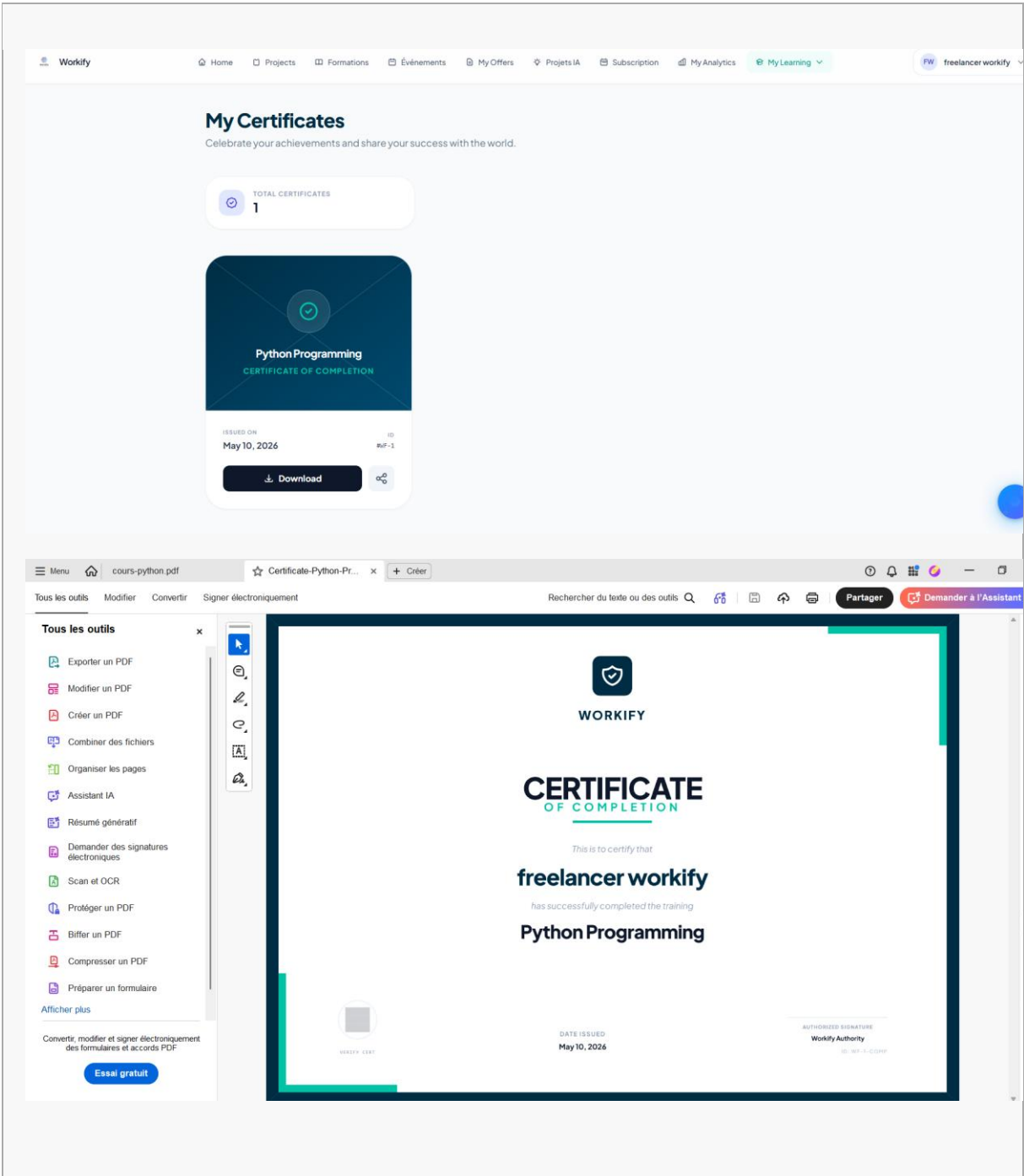
ESPRIT | Nour · Moatez · Safa · Louay · Oumaima | Page 48

The image displays two screenshots of the Workify platform interface, showing a course titled "Python Programming".

Top Screenshot: The interface shows the "Introduction" video player. The video title is "Learn Python in Less than 10 Minutes for Beginners (Fast & Easy)". The video player displays the text "Python 10 minute Crash Course" with the Python logo. The progress bar indicates "YOUR PROGRESS 0%". The "Mark Done" button is visible.

Bottom Screenshot: The interface shows the text-based lesson content. The lesson title is "Introduction". The content describes the Python interpreter's help function and provides examples of how to use it. The progress bar indicates "YOUR PROGRESS 100%". The "Lesson Completed" button is visible, along with a "Take Final Quiz" button.





Implementation — Pack & Subscription

[Home](#) [Browse Projects](#) [Find Freelancers](#) [Subscription](#) [My Projects](#)

freelancer Test

FREELANCER

Workify - 7 day(s) left

Logout

Choose Your Plan

Freelancer Subscription Plans

Unlock your full potential with our flexible subscription plans designed for freelancers at every stage of their career

CHOOSE A DURATION

1 month

3 months

6 months

1 year

Free Starter

A perfect entry plan to explore the platform, create your profile, and start interacting with projects or freelancers with basic access.

0 TND

/1 month

MONTHLY ~0.00 TND

✓ Create profile

✓ Limited access to projects

✓ Apply to 3 projects / month

✓ Basic messaging

Activate Free →

Pro

Boost your visibility and land more projects with powerful tools designed for growing freelancers.

30 TND

/1 month

MONTHLY ~30.00 TND

✓ Apply to 20 projects / month

✓ Unlimited messaging

✓ Access to all projects

✓ Profile boost (higher ranking)

✓ Ratings & feedback access

✓ Basic analytics (views, response rate)

✓ Priority support

Get Started →

Pro Max

Scale your freelance business faster with advanced insights, higher exposure, and more opportunities.

49 TND

/1 month

MONTHLY ~49.00 TND

✓ Apply to 50 projects / month

✓ Strong profile boost (top search placement)

✓ Unlimited messaging

✓ Advanced analytics (conversion, performance)

✓ Advanced feedback insights

✓ Access to premium/high-value projects

✓ Verified Badge (Trusted Freelancer)

✓ Fast priority support

Get Started →

Workify

The ultimate freelancer experience—maximize visibility, credibility, and income with full platform power.

79 TND

/1 month

MONTHLY ~79.00 TND

✓ Unlimited project applications

✓ Maximum profile exposure (homepage + top ranking)

✓ Advanced messaging (priority delivery)

✓ Full analytics dashboard

✓ Access to training & certifications

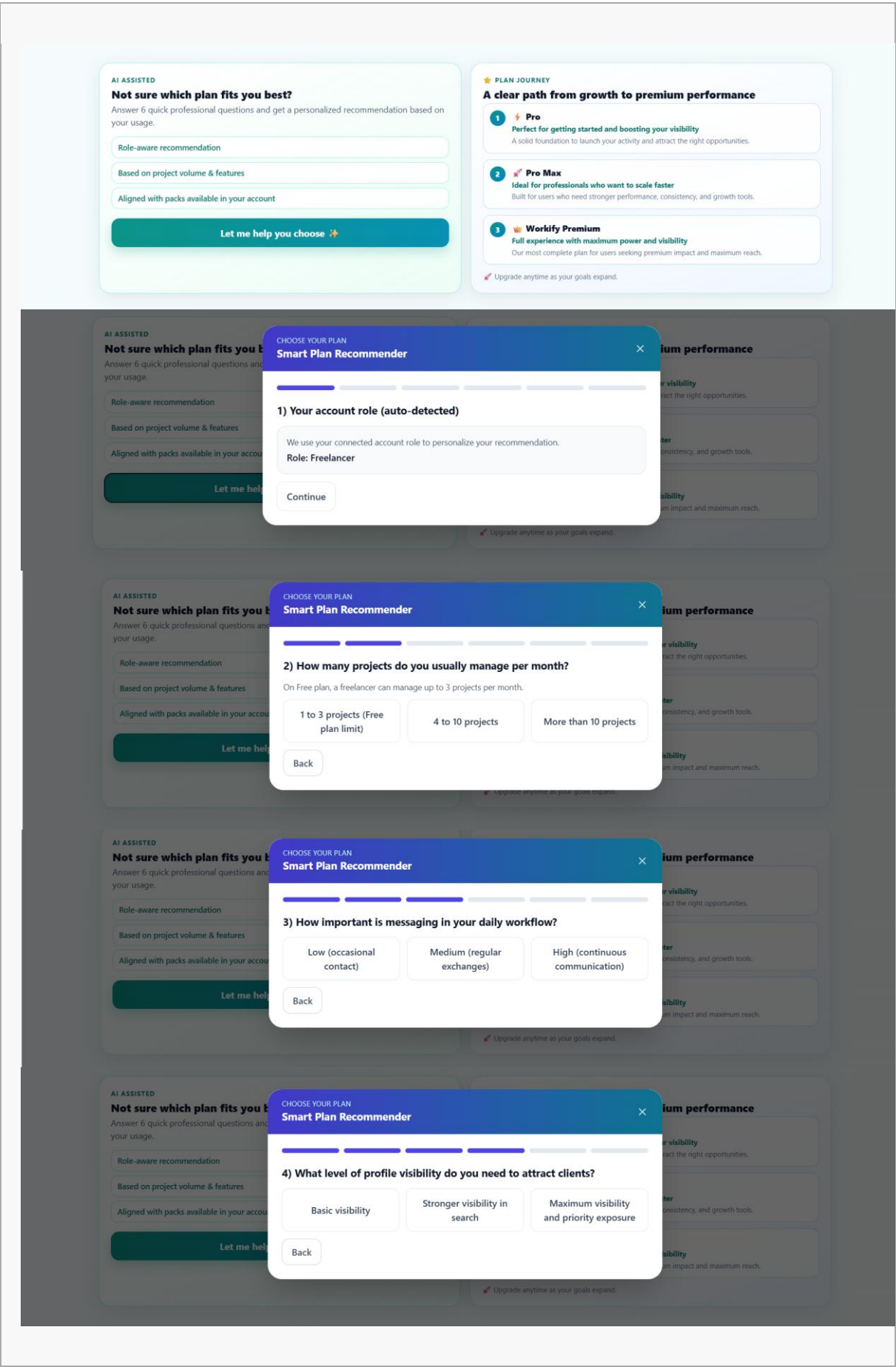
✓ Verified + Featured Freelancer Badge

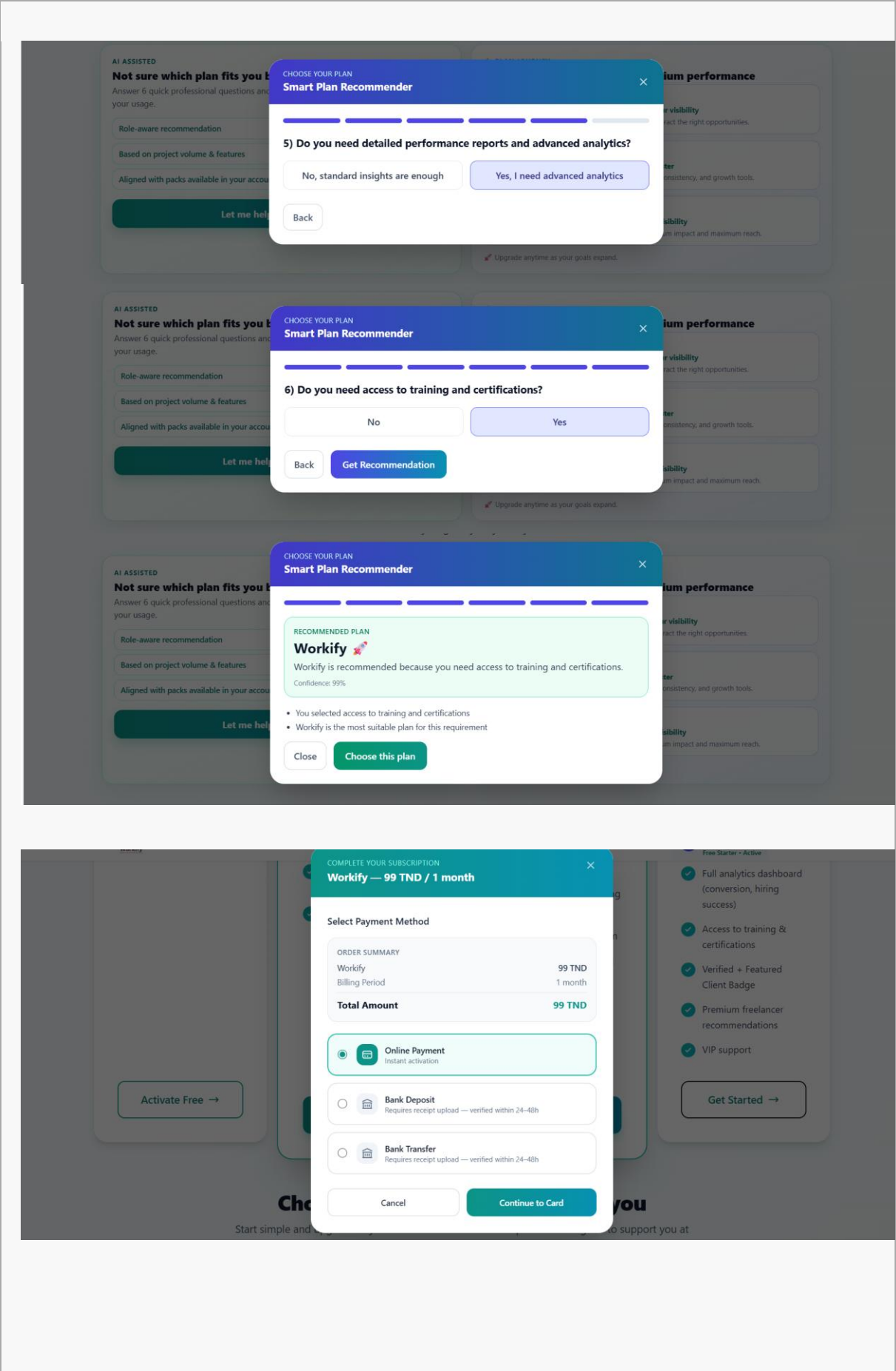
✓ Access to exclusive opportunities

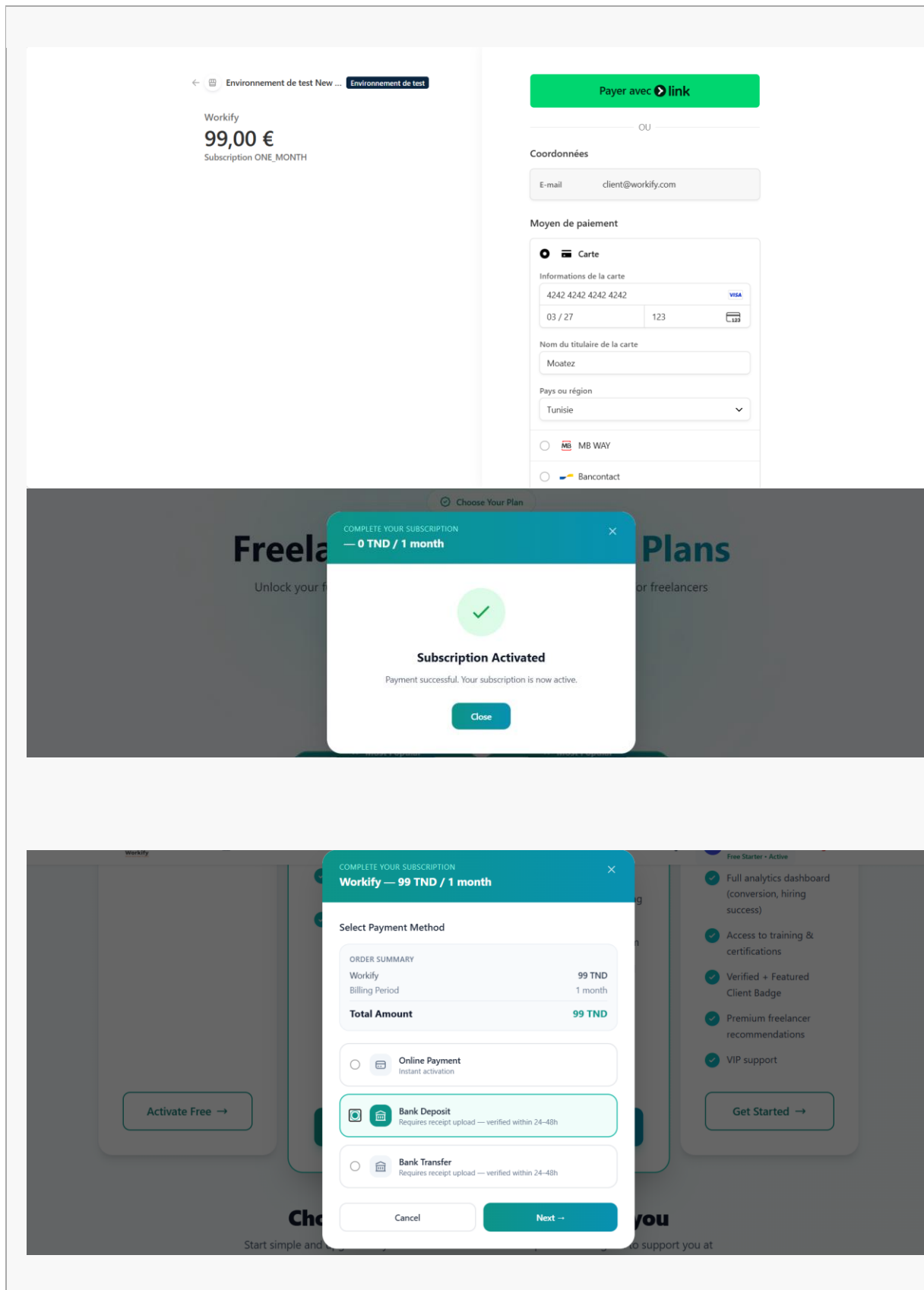
✓ VIP support

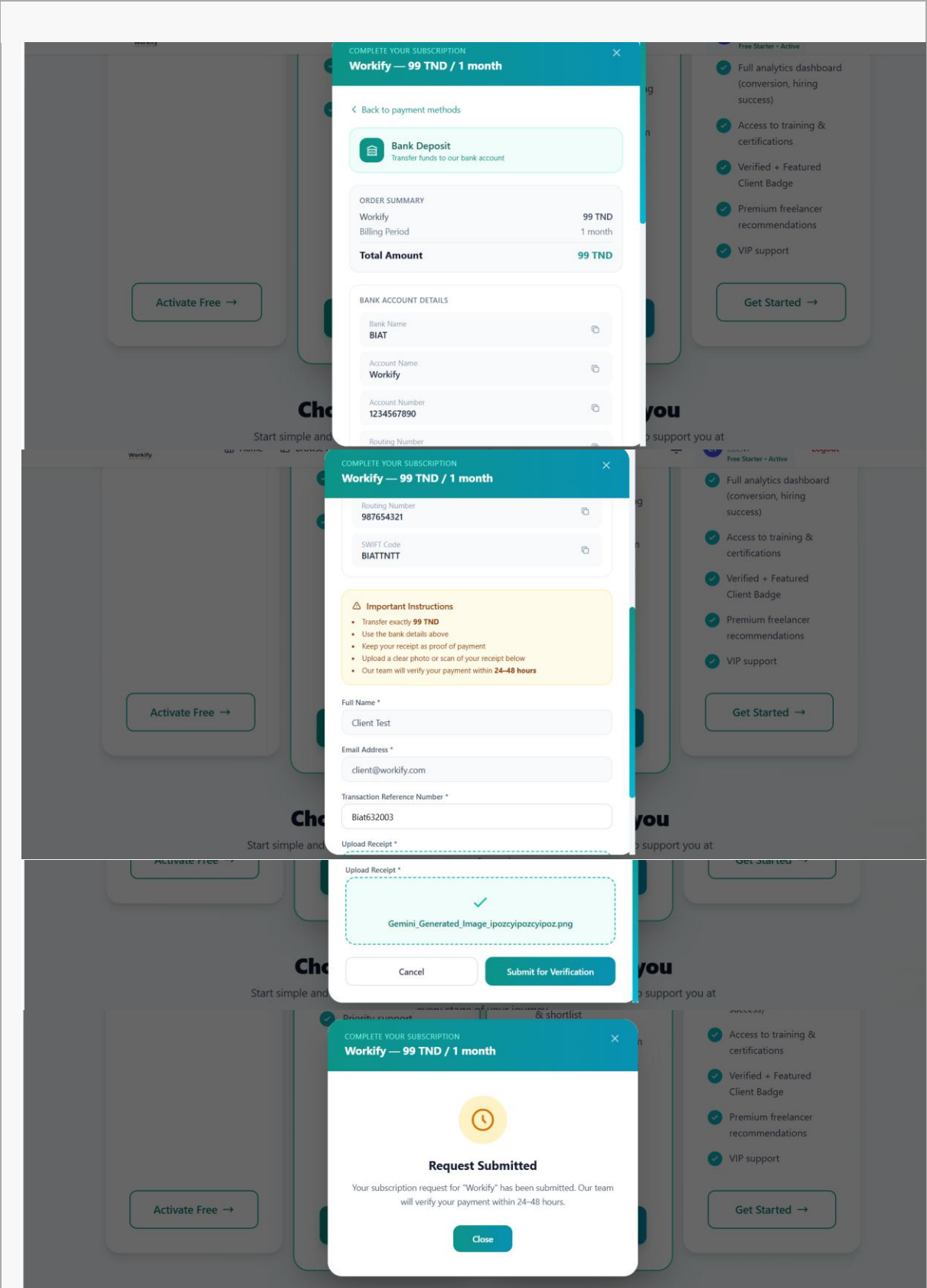
Get Started →

ESPRIT | Nour · Moatez · Safa · Louay · Oumaima | Page 52

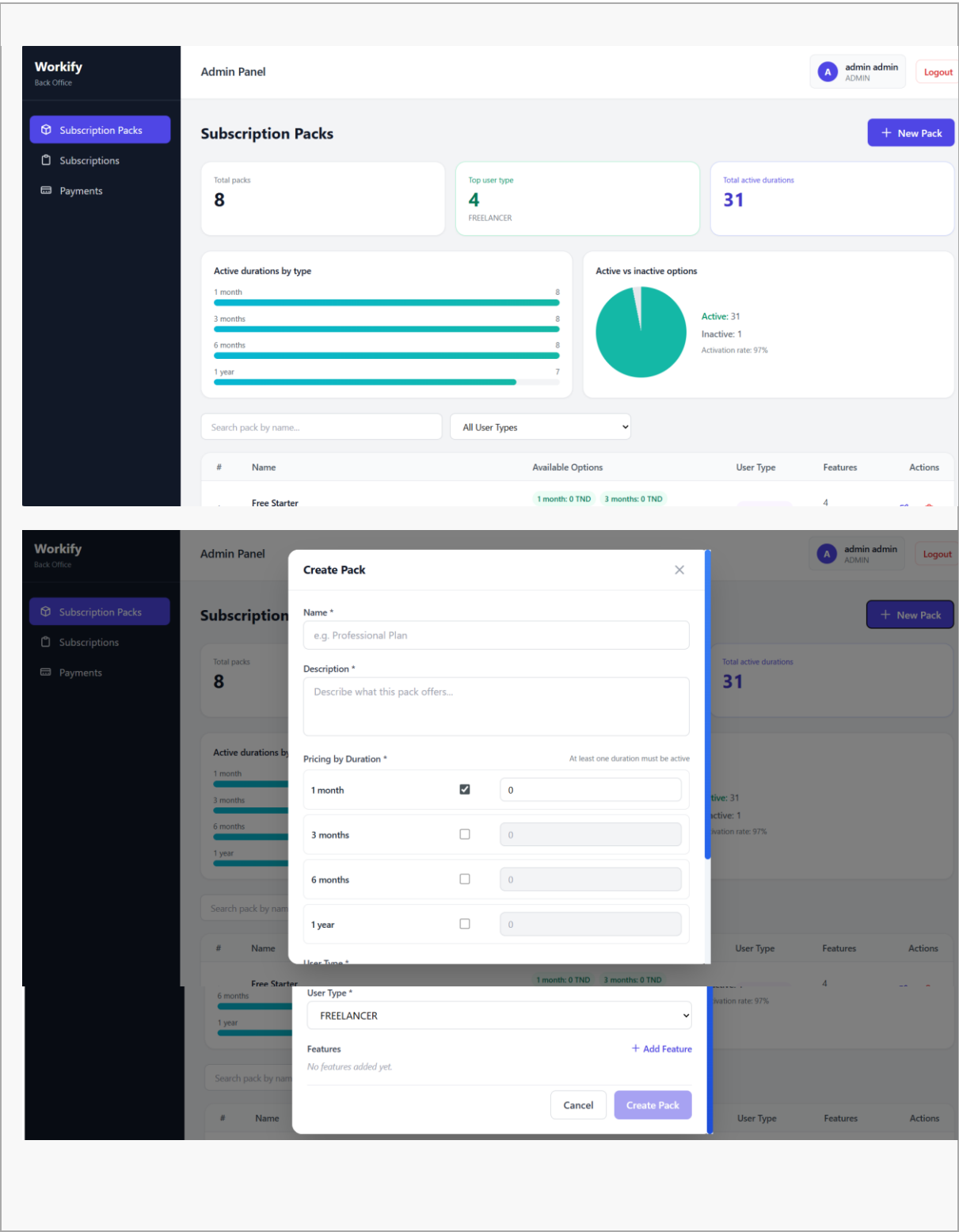


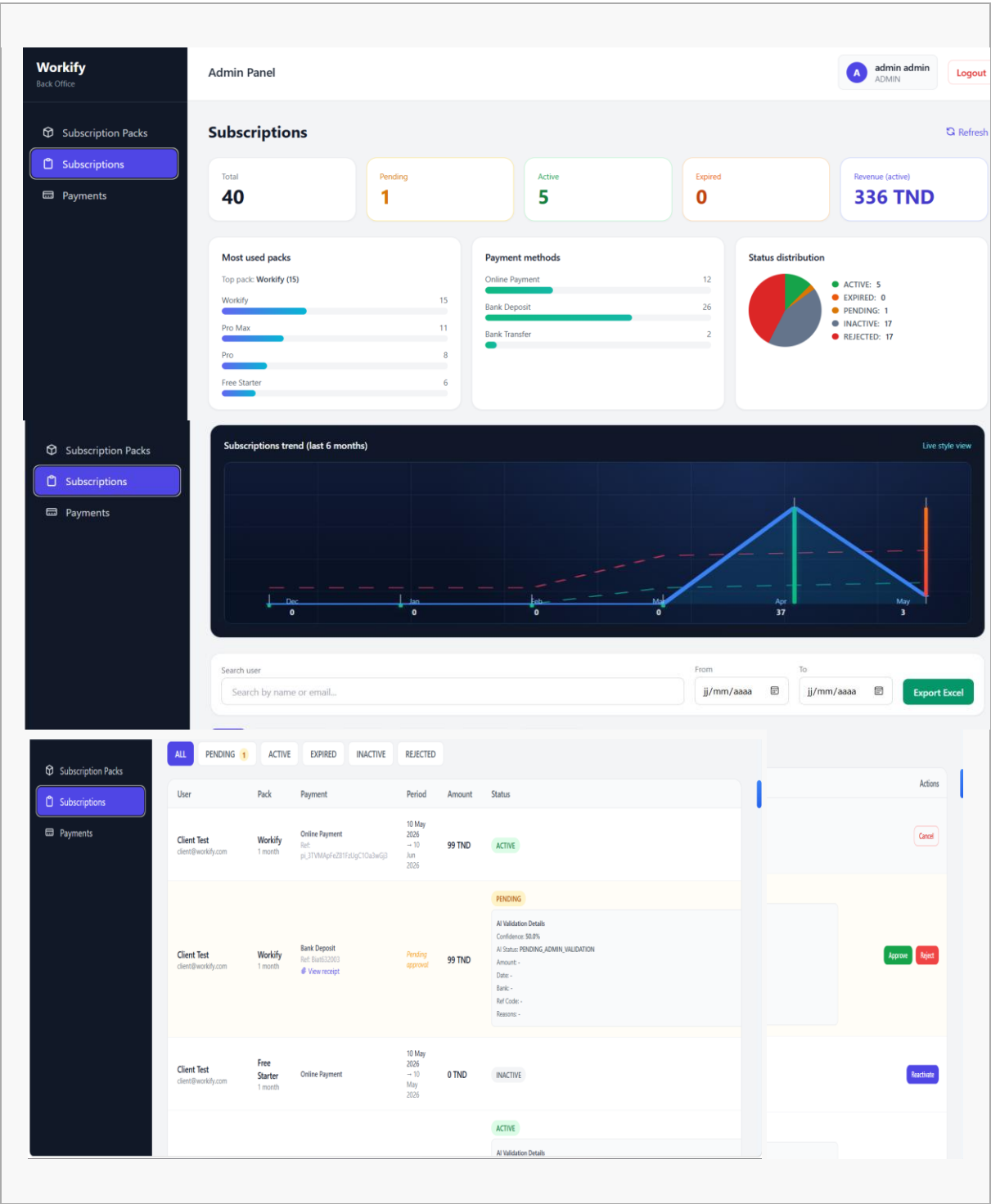






[Figure 36 : Pack selection and subscription interface]





Status distribution

ACTIVE: 5

EXPIRED: 0

PENDING: 1

INACTIVE: 17

REJECTED: 17

Subscriptions trend (last 6 months)

Live style view

Dec

0

Jan

0

Feb

0

Mar

0

Apr

37

May

3

Search user

From

To

Search by name or email...

jj/mm/aaaa

jj/mm/aaaa

Export Excel

ALL

PENDING

ACTIVE

EXPIRED

INACTIVE

REJECTED

User	Pack	Payment	Period	Amount	Status
Client Test client@workify.com	Workify 1 month	Online Payment Ref: p13TVMApf428Tf4lgC10a3wG3	10 May 2026 → 10 Jun 2026	99 TND	ACTIVE
Client Test client@workify.com	Workify 1 month	Bank Deposit Ref: Ban632003 View receipt		99 TND	PENDING
Client Test client@workify.com	Free Starter 1 month	Online Payment	10 May 2026 → 10 May 2026	0 TND	INACTIVE

Cancel

Approve

Reject

Reactivate

ESPRIT | Nour · Moatez · Safa · Louay · Oumaima | Page 58

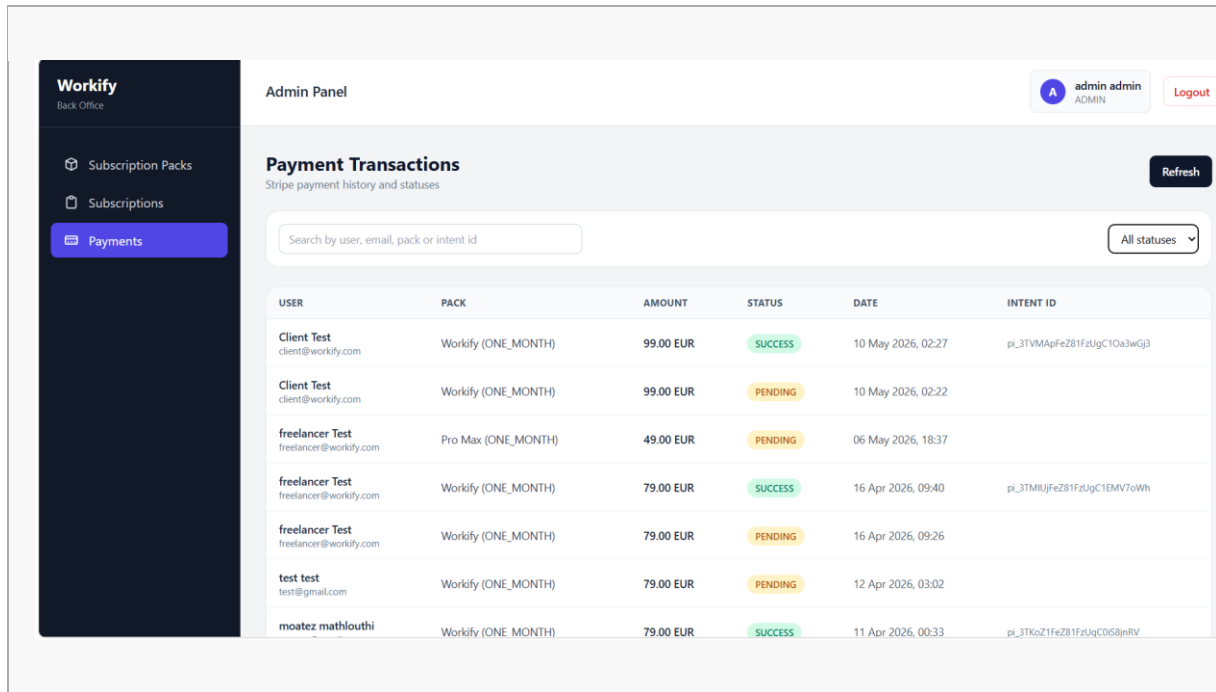


Figure 20 : Pack selection and subscription interface

4.6 DevOps Pipeline

The DevOps pipeline for the Workify platform follows the GitOps philosophy. It is defined via a Jenkinsfile versioned in the Git repository and is automatically triggered on each push to the main branches (main, develop).

Stage	Tool	Description	Duration
Checkout	Git / GitHub	Clone the repository and checkout the target branch	< 1 min
Build	Maven / Angular CLI	Compile and package the back-end and front-end applications	3-5 min
Unit Tests	JUnit / Jasmine	Execute all unit and integration tests	2-4 min
SonarQube Analysis	SonarQube	Static analysis of source code quality	2-3 min
Quality Gate	SonarQube	Verify all quality thresholds are met	< 1 min
Docker Build	Docker	Build optimized multi-stage Docker images for front and back	3-5 min

Stage	Tool	Description	Duration
Docker Push	Docker Hub	Publish the new images to the container registry	1-2 min
Deploy to K8s	Kubernetes	Apply Kubernetes manifests and roll out the new version	1-2 min

Table 13 : Jenkins CI/CD pipeline stages

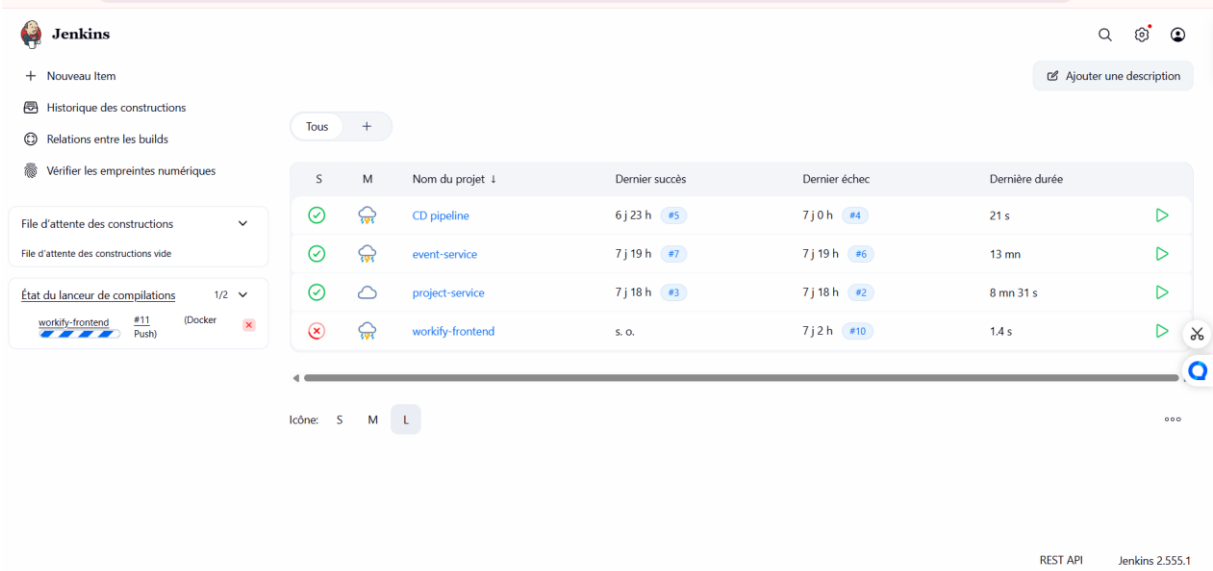


Figure22: Jenkins pipeline

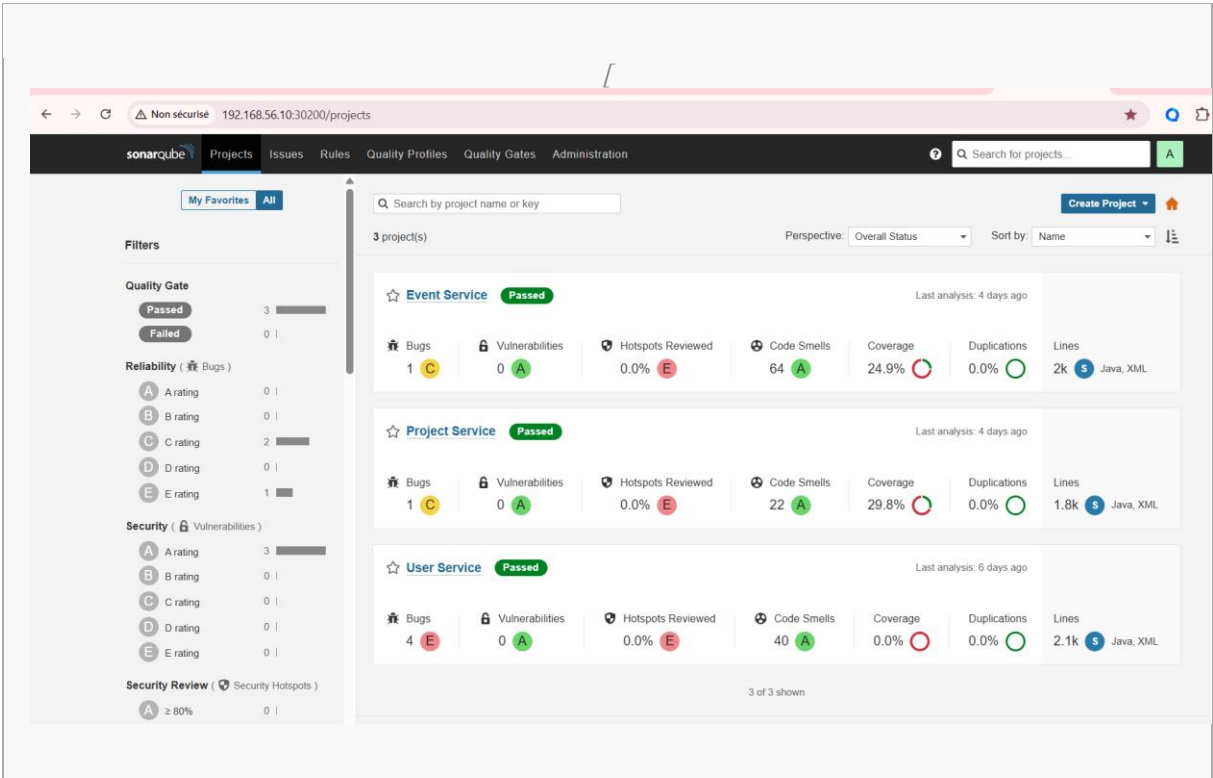


Figure 23 : SonarQube dashboard — Quality Gate Passed

```

vagrant@devops-smartfree: ~$ curl -s http://192.168.56.10:30080/
{"http_code": 200}
vagrant@devops-smartfree: ~$ curl -s http://192.168.56.10:30200/
{"http_code": 200}
vagrant@devops-smartfree: ~$ curl -s http://192.168.56.10:30300/
{"http_code": 302}
vagrant@devops-smartfree: ~$ curl -s http://192.168.56.10:8080/
{"http_code": 403}
vagrant@devops-smartfree: ~$ curl -s http://192.168.56.10:30090/
{"http_code": 302}
vagrant@devops-smartfree: ~$ kubectl get pods -n workify | grep -v Terminating
NAME                                READY   STATUS    RESTARTS   AGE
api-gateway-59f54495d5-7kkmd        1/1     Running   4 (5m43s ago)    4h
event-service-7f4559cd4b-h9nrl      1/1     Running   23 (77s ago)     4h
grafana-54bd456c87-99sgs            1/1     Running   4 (5m43s ago)    4h
keycloak-64cf469b6d-ttp24           1/1     Running   5 (5m43s ago)    4h
mysql-5c56f84c7-9kpbh               1/1     Running   17 (103s ago)    4h
node-exporter-8l78x                 1/1     Running   3 (5m43s ago)    2h
project-service-689496dfbd-lnv6b    0/1     ErrImagePull 20 (12s ago)     4h
prometheus-588dc78996-5rgn4         1/1     Running   2 (5m43s ago)    2h
sonarqube-69b5694584-79dxh          1/1     Running   3 (5m43s ago)    2h
user-service-5496cbfd68-zvdgz       1/1     Running   11 (80s ago)     4h
workify-frontend-5bcd885b57-qmwjw   1/1     Running   5 (5m43s ago)    4h
vagrant@devops-smartfree: ~$

```

Figure24: Kubernetes dashboard — Pods running in production

5. Versioning with GitHub

Throughout the development of the Workify platform, our team of five members adopted Git as the version control system and GitHub as the remote repository hosting platform. A structured branching strategy was implemented to ensure code quality, prevent conflicts and facilitate parallel development across the ten modules.

5.1 Branching Strategy

We followed the Git Flow branching model, organized as follows :

Branch	Purpose
main	Production-ready code — deployed automatically via the Jenkins pipeline
develop	Integration branch — all features are merged here before release
feature/module-name	One dedicated branch per module for isolated development (e.g. feature/user-management, feature/ai-matching)

Branch	Purpose
hotfix/issue-name	Emergency fixes applied directly to main when critical bugs are detected in production
sprint/sprint-number	Sprint-level branch grouping all features of a given sprint (e.g. sprint/sprint-1)

Table 14 : Git branching strategy

5.2 Commit Convention

To maintain a readable and traceable commit history, our team followed the Conventional Commits standard :

Prefix	Usage	Example
feat:	New feature added	feat: add AI matching algorithm
fix:	Bug fix applied	fix: resolve JWT token expiration issue
refactor:	Code restructuring without behavior change	refactor: optimize feedback fraud detection
docs:	Documentation update	docs: update API endpoint documentation
test:	Unit or integration test added	test: add JUnit tests for subscription module
chore:	Build or configuration changes	chore: update Docker base image to JRE 17
ci:	CI/CD pipeline changes	ci: add SonarQube quality gate to Jenkinsfile

Table 15 : Commit convention

5.3 Pull Request Workflow

Every new feature followed a strict Pull Request (PR) workflow before being merged into the develop branch :

- The developer creates a feature branch from develop following the naming convention
- The developer commits his changes following the Conventional Commits standard
- The developer opens a Pull Request targeting develop with a clear title and description
- At least one team member reviews the code and approves the Pull Request
- The CI pipeline (Jenkins) runs automatically : build, tests and SonarQube quality analysis
- If the Quality Gate passes, the Pull Request is merged into develop
- The feature branch is automatically deleted after successful merging

6. Work Environment

In this section, we detail the tools necessary for the realization of the application, including the programming languages, development environments, frameworks and DevOps tools used.

6.1 Hardware Environment

During the realization of our application, we used computers having the following characteristics :

Component	Specification
Machine	Personal Laptop
Processor	Intel Core i7 / AMD Ryzen 7 — 2.40 GHz
RAM	16.0 GB
Storage	512 GB SSD
Operating System	Windows 11 / Ubuntu 22.04 — 64-bit system

Table 16 : Hardware environment

6.2 Software Environment

In this section, we present the tools and software that enabled the realization of our application :

Software / Tool	Description
	Open-source front-end framework based on TypeScript, used to build the Single Page Application (SPA) of the Workify platform with a responsive and intuitive user interface.
	Open-source Java framework that facilitates the creation of REST APIs and microservices. Used for the entire back-end layer, business logic and security management.
	Relational database management system used for persistent and structured storage of all platform data.
	Security framework combined with JSON Web Tokens for secure authentication, authorization and role-based access control (RBAC) across all platform actors.

Software / Tool	Description
	Protocol enabling real-time bidirectional communication between client and server. Used for the instant messaging module and live notifications.
	Containerization platform used to package and isolate each application service in an independent, portable container.
	Container orchestration system used for production deployment, automatic horizontal scaling and high availability management.
	Open-source automation server used to build, test and deploy the application through a fully automated 8-stage CI/CD pipeline.
	Continuous code quality inspection platform used to detect bugs, security vulnerabilities and code smells through static source code analysis.
 IntelliJ IDEA	Integrated development environment (IDE) used for Spring Boot back-end development.
	Lightweight and powerful source code editor used for Angular front-end development.
	API testing tool used to build, test, document and validate all REST API endpoints.
	MySQL est système de gestion de bases de données relationnelles
	Source control platform used for version management, Git Flow branching strategy and team collaboration via Pull Requests.

Software / Tool	Description
	UML modeling tool used to design use case diagrams, class diagrams and sequence diagrams.

Table 17 : Software environment

Conclusion

In this chapter, we began by presenting the methodology for the sprint workflow. We detailed the product backlogs, use case diagrams and sequence diagrams of each sprint, as well as the corresponding interfaces. We then presented the versioning strategy adopted using GitHub with the Git Flow model and the Conventional Commits standard. Finally, we presented the hardware environment, the software environment and the tools used for the realization of our application. The Workify platform has been successfully implemented across four sprints, delivering a complete, modular and production-ready solution.

General Conclusion

In our project entitled "Workify — Smart Freelance & Project Matching Platform", we developed a complete web application to address the limitations of existing freelancing platforms. We began by establishing the project context, highlighting the problems faced by freelancers, clients and partners in the current digital ecosystem. We then carried out a comparative analysis of similar solutions in order to identify their limitations and define the problem statement, before proposing a comprehensive and adapted solution.

We subsequently detailed the functional and non-functional specifications, as well as the complete product backlog of our solution. The sprint planning, the identification of the global use case diagram and the class diagram were addressed in the second chapter.

The last chapter was devoted to the application of the Agile Scrum method, beginning with the analysis of the product backlog of each sprint, the design of use case and sequence diagrams, and the illustration of the implemented interfaces for each sprint. The complete DevOps pipeline — including Docker containerization, Jenkins CI/CD automation with 8 stages, SonarQube code quality analysis and Kubernetes production deployment — was successfully implemented in Sprint 3. A structured versioning strategy using the Git Flow model and GitHub was adopted throughout the entire project.

Through this project, our team of five members acquired valuable technical and collaborative skills. On the theoretical level, this project enriched our expertise in several domains : mastery of the Agile Scrum methodology, proficiency in Spring Boot and Angular frameworks, understanding of artificial intelligence integration in web applications, and knowledge of DevOps practices including containerization, continuous integration and continuous deployment.

On the professional level, this project allowed us to experience the realities of software development in a team environment, with its constraints and satisfactions.

As future work, our application can be further developed and improved through the addition of several other features. These include a Deep Learning model (BERT) for more accurate semantic matching, a native mobile application for iOS and Android, an identity and skills

verification system through integrated technical assessments, international expansion with multi-currency and multi-language support, and an advanced analytics dashboard with market predictions using Machine Learning.

Bibliography

- [1] : Atlassian, "What is Scrum?", <https://www.atlassian.com/agile/scrum>
- [2] : Angular documentation, <https://angular.io/docs>
- [3] : Spring Boot documentation, <https://spring.io/projects/spring-boot>
- [4] : JSON Web Token (JWT) — RFC 7519, <https://jwt.io/introduction>
- [5] : Docker documentation, <https://docs.docker.com>
- [6] : Kubernetes documentation, <https://kubernetes.io/docs>
- [7] : Jenkins documentation, <https://www.jenkins.io/doc>
- [8] : SonarQube documentation, <https://docs.sonarqube.org>
- [9] : MySQL documentation, <https://dev.mysql.com/doc>
- [10] : WebSocket protocol — RFC 6455, <https://datatracker.ietf.org/doc/html/rfc6455>
- [11] : Twilio API documentation, <https://www.twilio.com/docs>
- [12] : Google Calendar API documentation, <https://developers.google.com/calendar>
- [13] : Conventional Commits specification, <https://www.conventionalcommits.org>
- [14] : Git Flow branching model, <https://nvie.com/posts/a-successful-git-branching-model>
- [15] : StarUML documentation, <https://docs.staruml.io>
- [16] : Figma documentation, <https://help.figma.com>